

Review Article

The role of AI in shaping educational experiences in computer science: A systematic review

Anahita Golrang^{a,*}, Kshitij Sharma*Department of Computer Science, Norwegian University of Science and Technology, 7034 Trondheim, Norway*

ARTICLE INFO

Keywords:

Computer science education
Learning analytics
Educational data mining
Artificial intelligence
Higher education
Undergraduate
Novice programming

ABSTRACT

The integration of artificial intelligence (AI) in computer science education (CSE) has earned significant attention due to its potential to enhance learning experiences and outcomes. This systematic literature review provides one of the first domain-specific and methodologically robust syntheses of AI applications in undergraduate CSE. Through a comprehensive analysis of 40 peer-reviewed studies, we offer a fine-grained categorization of course contexts, AI methods, and data types. Our findings reveal a predominant use of supervised learning, ensemble methods, and deep learning, with notable gaps in generative and explainable AI. The review highlights the post-pandemic increase in AI-driven programming education and the growing recognition of AI's role in addressing educational challenges. This study offers technical and pedagogical insights that inform future research and practice at the intersection of AI and computer science education.

1. Introduction

The adoption of artificial intelligence (AI) in higher education (HE) has significantly accelerated recently (Chu et al., 2022), leading to the introduction of numerous new AI tools. Chen et al. examined the benefits of AI for both educators and students in HE contexts. These advantages include **employing AI in HE** to: customize teaching for various learner profiles (Verdú et al., 2017) provide individualized and timely feedback (Dever et al., 2020), design assessments (Baykasoğlu et al., 2018), and predict student academic outcomes (Çağataylı & Çelebi, 2022). This body of research informs educators about the practical implementations of artificial intelligence in higher education (AIED) (Crompton & Burke, 2023).

AI has become an increasingly central field in computer science and is transforming the way we live, work, and learn. In the context of undergraduate computer science education, the integration of AI tools has generated significant interest due to their potential to improve teaching and learning outcomes for both students and educators. Over the past decade, a growing body of literature has examined the incorporation of AI into computer science education, addressing topics such as its curricular impact, the effectiveness of AI-supported pedagogy, and the challenges surrounding implementation. One key insight from existing research is the demonstrated ability of AI-based approaches to enhance student learning by fostering greater engagement, motivation, and critical thinking. For instance, educational games and simulations driven by AI have proven particularly effective for teaching complex

concepts such as algorithms and data structures in a way that is both interactive and engaging (Liu, 2023).

Moreover, the rising demand for Information Technology (IT) roles has sharpened the focus on education in Computer Programming and related fields. Recent trends indicate that STEM disciplines science, technology, engineering, and mathematics increasingly demand fundamental IT skills and knowledge. Central to any IT-centric educational program is a sequence of programming courses. Thus, improving student engagement and learning outcomes in these courses is essential (Mai et al., 2022). Despite the escalating need for computer science (CS) experts, there is still a notable deficit of individuals with at least a Bachelor's degree in many IT-related jobs. To tackle this shortage of qualified professionals, strategies could include measures to lower the dropout rates from CS degree programs. It is generally more cost-effective to retain existing students than to recruit new ones (Naseem et al., 2022). This highlights the importance of preserving the quality of computer science education to develop competent graduates while striving to reduce dropout rates as much as possible.

Despite the critical importance of these skills, there is a significant dropout rate in introductory programming courses offered during the initial years of undergraduate studies (Naseem et al., 2022). Computer science students in Europe and the US experience some of the highest dropout rates across all undergraduate fields. According to the Higher Education Statistics Agency (HESA), the dropout rate for computer science undergraduates in the UK was 9.8% for the 2016/17 academic

* Corresponding author.

E-mail addresses: Anahita.m.golrang@ntnu.no (A. Golrang), kshitij.sharma@ntnu.no (K. Sharma).<https://doi.org/10.1016/j.chbah.2025.100199>

Received 9 July 2024; Received in revised form 20 August 2025; Accepted 22 August 2025

Available online 13 September 2025

2949-8821/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

year, the highest among all disciplines at that time. Similarly, Debut (2020) also reported a 9.8% dropout rate for computer science, identifying it as the discipline with the highest dropout rate. In the US, the dropout rate for computer science majors was even higher, at 10.7%, mainly due to the demanding nature of the program which encompasses complex technical coursework and extensive mathematical requirements. These statistics emphasize the demanding conditions of computer science programs and point to a need for improved preparatory guidance for undergraduate computer science students and stronger support systems within educational institutions (Higher Education Statistics Agency (HESA), 2017; Kilroy, 2022). To bridge this gap, this survey primarily aims to investigate the application of artificial intelligence (AI) in the field of computer science education (CSE), with a focus on undergraduate studies.

Contributions: This systematic literature review makes three significant contributions to the field of AI in computer science education (CSE).

- (1) To the best of our knowledge, it is the first domain-specific review to synthesize artificial intelligence applications exclusively within undergraduate CSE, addressing a critical gap where prior works focused on broader STEM education or isolated subdomains (e.g., database instruction).
- (2) It establishes a rigorous, empirically grounded taxonomy by categorizing AI methods (e.g., supervised learning, ensemble techniques), data sources (e.g., log traces, academic performance), and course contexts (e.g., programming, hardware, theory), while quantitatively revealing a pandemic-driven acceleration in publications.
- (3) It identifies six pivotal research themes: learning behavior, Learning performance outcomes, At-risk detection, Interventions, Hints or recommendations, and Automatic content extraction, and underscores underexplored opportunities (e.g. explainable AI) alongside ethical imperatives (e.g., bias mitigation, transparency). Collectively, this work provides educators with evidence-based strategies for AI-enhanced pedagogy and equips researchers with a roadmap for future studies on personalization, equity, and longitudinal efficacy in CSE.

To address this objective, we have formulated four primary research questions. The first research question (RQ1) explores the artificial intelligence methods and type of data. In RQ2, we shift our focus to the educational context, aiming to identify the courses and technologies that have been utilized in these studies. The third research question (RQ3) is dedicated to examining the objectives and findings of the state-of-the-art research in this domain. RQ4 assesses any notable shifts in publishing trends before and after the COVID-19 pandemic. These primary research questions are further subdivided into the following sub-research questions:

RQ1: What Artificial intelligence methods and data types have been reported in undergraduate CSE research?

RQ1a: Which artificial intelligence methods have been applied in the state-of-the-art?

RQ1b: Which type of data have been collected in this field?

RQ2: How is the educational context in terms of targeted courses and the technologies employed?

RQ2a: Which courses have been targeted?

RQ2b: Which technologies have been used?

RQ3: What are the research objectives and findings in the current state-of-the-art?

RQ4: Are there any notable shifts in publishing trends before and after the COVID-19 pandemic?

RQ5: What are the opportunities and challenges in using AI in undergraduate CSE?

The rest of this paper is structured as follows, Section 2 looks into the review articles published in the CSE domain, this section is

written with the objective in mind to ensure the contribution criteria of the paper. Section 3 is dedicated to the methodology which has been followed to collect related articles including the Search query construction, Inclusion, exclusion, and quality assurance criteria of the papers. In Section 4, the most significant findings of the SLR as well as the answers to the research questions are reflected. In Section 5, the current challenges, and opportunities for future works are discussed in detail.

2. Related work & background

In this section, we first list some of the most pertinent problems and how AI can help solve them. Then, we present a brief review of the other SLRs in the field to establish the gaps and requirement of the present SLR.

2.1. Current challenges in CS education

Undergraduate computer science education is **conceptually demanding and abstract**. Core topics such as algorithms, data structures, computational theory, and software architecture require high levels of abstraction and logical reasoning, which many beginners struggle to master without hands-on, adaptive support (Chandrasekara et al., 2025; Colburn & Shute, 2007; Qu et al., 2019).

Student cohorts in CS programs are highly heterogeneous. There is a wide range of students enrolled in computer science programs, from those with extensive prior coding experience to total beginners. Due to differences in mathematical skills, logical reasoning, and exposure to technology, students have a wide range of learning needs (Espiritu & Todorovic, 2020; Ibe et al., 2018; Wang & Hejazi Moghadam, 2017).

Keeping curricula current with industry and technological change poses an ongoing challenge. As programming languages, tools, frameworks, and industry practices continue to evolve at a rapid rate, it has become increasingly difficult for curricula to remain up to date with the changes. To ensure that learning materials remain relevant, educators and institutions are constantly under pressure to make sure they are constantly updated (Abildinova et al., 2024; Pirker et al., 2020; Randolph et al., 2008; Zhu & Zhang, 2025).

Large class sizes and unequal access to resources further limit effective instruction. Introductory CS courses often prevent instructors from providing individualized attention. Moreover, not every student has equal access to computer resources, which contributes to the widening of the digital divide among students (Goode et al., 2018; Margolis et al., 2015; Wiredu et al., 2024).

The iterative nature of programming requires timely feedback. However, in traditional settings, delays in grading or instructor responses hinder learning momentum (Elegbeleye & Isong, 2025; Rocha et al., 2023; Toll & Wingkvist, 2017).

High attrition rates remain a persistent challenge. There is often a high attrition rate among students enrolled in CS courses. It has been shown that students who feel disconnected, lost, or under-supported are more likely to disengage and make a decision to drop out, especially in online or hybrid learning environments (Obaido et al., 2023; Ross, 2025; Takács et al., 2022).

Finally, **ethical and equity concerns arise with the increased use of AI and analytics in education.** With the increasing application of AI and analytics in educational decision-making, there have been concerns raised about algorithmic bias, transparency, and data privacy as a result. It is imperative that these issues are addressed so that inequities in CS education are not perpetuated.

2.2. How AI can address these challenges

AI can personalize learning and adapt instruction to individual students. It is possible to tailor content difficulty, pacing, and instructional strategies based on the individual profiles of students using artificial intelligence-powered adaptive learning platforms. Students can master complex CS concepts at their own pace with the help of Intelligent Tutoring Systems (ITS), which provide step-by-step guidance (Bassner et al., 2024; Francisco & de Oliveira Silva, 2022; Gaitantzi & Kazanidis, 2025; Nesbit et al., 2014).

AI enables customized learning paths for diverse learners. In order to recommend customized learning paths, artificial intelligence can analyze student backgrounds and performance patterns. It is possible to provide beginners with foundational tutorials, while more advanced learners can receive more challenging tasks. In this way, an equitable progression can be ensured (Awasthi & Shukla, 2025; Liu, 2023).

AI supports curriculum modernization and alignment with industry needs. Artificial intelligence can be used to scan job markets, research publications, and emerging technologies to suggest curriculum updates based on the use of Natural Language Processing (NLP) and data mining. As a result of this, students will be able to gain real-world skills that are aligned with the demands of the workplace (Coroiu et al., 2024; Franklin et al., 2025; Liu et al., 2024; Yee-King et al., 2020).

AI provides immediate feedback through automated code analysis. There is a new generation of AI-driven automated code analysis tools that are capable of detecting syntax errors, logic flaws, and optimization suggestions and offering instant feedback on programming assignments. By doing this, instructors are able to reduce their workload while supporting iterative learning (Fang et al., 2024; Plate & Hutson, 2025).

AI-powered interactive tools enhance engagement and motivation. AI-powered chatbots, virtual assistants, and gamified learning environments are able to maintain continuous interaction with students, providing motivation, answering queries, and keeping them engaged, particularly in online settings, where they can provide continuous interaction and motivation (Chen & Chang, 2024; Gaitantzi & Kazanidis, 2025; Senanayake et al., 2024).

AI facilitates early intervention and supports ethical educational practices. As a result of predictive analytics and machine learning models, you can monitor student engagement, performance, and behavioral patterns in order to identify students who may be at risk of disengagement or failure. It is then possible for the institutions to offer timely interventions, such as tutoring, counseling, or additional support, when needed. The Explainable AI (XAI) framework enables educators and students to gain a better understanding of AI-driven decisions by making them transparent and understandable. In this way, trust can be built, bias can be minimized, and ethical use of AI in educational settings can be ensured. The privacy of sensitive student information can also be protected through privacy-preserving techniques such as federated learning (Chaushi et al., 2023; Khosravi et al., 2022; Marconi & Bandini, 2021; Rachha & Seyam, 2023; Raees et al., 2024; Zoppis et al., 2020).

2.3. Challenges and new needs brought by COVID-19

The COVID-19 pandemic triggered a rapid transformation in computer science (CS) education. It prompted widespread investigations into remote teaching strategies, digital tool adoption, and equity-focused practices (McGill et al., 2022; Mouza et al., 2022; Toti et al., 2025). In one study surveying 137 students and 10 instructors after a full year of remote teaching, sentiment shifted markedly—students reported improved experience over time and a growing openness to continuing at least some instruction online, though concerns remained about its effectiveness for early-years learners and instructors' worries about learning outcomes (McGill et al., 2022; Mouza et al., 2022; Toti et al., 2025).

Researchers have documented the initial severe challenges CS students faced during the abrupt shift to emergency remote teaching, including disruptions in hands on learning and collaboration, followed by gradual adaptation and increased satisfaction over time (Siegel et al., 2022; Toti et al., 2025). Additional large-scale research from Poland found that, while the transition enhanced flexibility, students actually maintained or improved academic performance during online and hybrid courses—suggesting that online formats, when well designed, can support learning outcomes effectively (Casper et al., 2022; Charytanowicz et al., 2024; McGill et al., 2022).

The pandemic also accelerated research into pedagogical innovation. Studies examined emergency remote teaching (ERT) practices, the effectiveness of asynchronous vs synchronous instruction, and the design of virtual labs, auto graders, and cloud-based coding platforms. The integration of computer supported collaborative learning (CSCL) approaches—once primarily used in face to face contexts—was rapidly adapted to virtual pair programming, group projects, and online peer feedback environments (Haleem et al., 2022).

Equity, diversity, and inclusion (EDI) emerged as central themes in pandemic-era CS education research. Practitioner-focused work using the CAPE framework (Capacity, Access, Participation, Experience) highlighted how COVID-19 disproportionately affected underserved and underrepresented students, exacerbating digital divide issues and highlighting the need for sustainable & equitable CS education systems (Chen et al., 2025; Huang et al., 2024; McGill et al., 2022; Toti et al., 2025). In response, researchers also investigated how teacher professional development (TPD) shifted online. A virtual PD institute studying CS teacher learning showed that remote delivery could enhance teacher confidence in CS content and pedagogy, although it also posed challenges related to workload and the emotional strain of the pandemic (Mouza et al., 2022).

Assessment and learning analytics gained renewed attention as well: researchers explored methods to prevent academic misconduct in online programming exams, while learning analytics tools became instrumental in monitoring student engagement, retention, and identifying learners at risk in remote environments—topics integrated into broader studies on digital learning trends (Charytanowicz et al., 2024; Haleem et al., 2022).

2.4. Other systematic literature reviews

Numerous systematic literature reviews have examined computer science education (CSE) from different perspectives. To confirm the contribution of this article, we evaluated 41 literature review papers using the keywords provided in Section 3. An overview of these papers is shown in Table 1. This table categorizes each review based on its focus on computer science education (CSE), and whether the educational content relates to higher education or undergraduate levels, K-12 schooling, or covers a broader range of educational levels. The last column of the table highlights the main focus of each paper, clearly showing that the first three rows are dedicated to the reviews most relevant to computer science education at the higher or undergraduate level. Next, we will provide summaries of these significant articles to further outline their contributions and how they relate to the topics discussed in the current review.

Automation in programming education has been a key focus of prior reviews. as discussed by Conceição et al. (2022). This review analyzed 119 articles from 2017 to 2021, exploring various aspects such as the volume of publications, their geographical origins, the venues where they were published, the Learning Management Systems (LMS) used, the programming languages taught, users' perceptions of the feedback, and the feedback types provided. The results show a growing research field with contributions from 37 countries, underscoring the global interest in this area. The primary programming languages discussed are Java, Python, C, and C++. Additionally, the review points

Table 1
State-of-the-art summary.

		CSE	Edu focus	Main focus
1	Conceição et al. (2022)	Yes	undergraduate	Automatic Feedback
2	Sim and Lau (2022)			Imperative-first or object-first programming
3	Calderon et al. (2023)			Active learning strategies applied in programming
4	Sanusi et al. (2023)	Yes	K-12	ML studies in k-12
5	Jia et al. (2023)	NO		AI in science education (AISE)
6	Prekaj et al. (2020)	Yes	Broad Scope	ML for student drop out in online education
7	Paiva et al. (2022)			Learning analytics included
8	Ng et al. (2023)			Pedagogy, learning and teaching to foster AI literacy
9	Silva et al. (2020)			Collaborative Learning (CSCL) in programming education
10	Varghese and Renumol (2023)			Digital games for computational thinking
11	Oleson et al. (2022)			Demographic in CER
12	Apiola et al. (2023)			Emerging topics in computing education research
13	Mihaescu and Popescu (2021)	Broad scope	Broad Scope	Quality and accessibility of datasets in EDM
14	Chen, Zou, Xie, Cheng, and Liu (2022)			AI applications in AIED
15	Wang and Mousavi (2023)			Log variables for student academic prediction
16	Hellas et al. (2018)			Student academic performance prediction
17	Kulik and Fletcher (2016)			Intelligent tutoring systems as an instructional tool
18	Elmoazen et al. (2023)			Learning analytics in virtual labs
19	Machado et al. (2021)			Adaptive Curriculum Sequencing by metaheuristics
20	Mallik and Gangopadhyay (2023)			AI driven methods in education process
21	Sypsas and Kalles (2022)			Educational process mining
22	Shaik et al. (2023)			Sentiment analysis in educational setting
23	Salau et al. (2022)			Recommender systems (RSs) goals, techniques, target user
24	Khan and Ghosh (2021)			EDM for in-class performance analysis and prediction
25	Ahmad et al. (2023)			AI strategies in AIED
26	Chen, Zou, and Xie (2022)			LA Structural topic modeling and bibliometric analysis
27	da Silva et al. (2021)			Intelligent services applied to distance learning
28	Alalawi et al. (2023)			ML to predict student performance
29	Guo et al. (2022)			Data-Driven educational anomaly analytics
30	Heikkinen et al. (2023)			LA interventions to enhance self-regulated learning
31	Li, Herbert, et al. (2022)	Broad scope	Higher Education	LA to identify retention factors in CSL environments
32	Algarni and Sheldon (2023)			Academic courses recommender algorithms
33	Berland et al. (2014)			EDM quantitative constructionist learning
34	Munir et al. (2022)			AI-based approached in higher education
35	McConvey et al. (2023)			Decision making algorithms in higher education
36	Zhong (2015)			LA in Higher education
37	Wong et al. (2018)			Patterns and trends in LA in higher education
38	Murtaza et al. (2022)			Requirements and challenges for personalized e-learning
39	Li et al. (2023)			Predictive Analysis for Student Admission
40	Albreiki et al. (2021)			ML in predicting student performance
41	Bond et al. (2024)			The scope and nature of AIED in higher education

out a prominent trend towards developing dedicated platforms for automatic feedback in programming education.

Pedagogical strategies in introductory programming have also received substantial attention. The systematic review in [Sim and Lau \(2022\)](#) addresses the debate between “imperative-first” versus “object-first” pedagogical approaches in introductory programming courses (CS1). While many educational and industrial institutions opt for Object-Oriented Programming (OOP) languages to teach CS1, the review suggests that an object-first approach is not necessary. Instead, it argues that the imperative-first approach better aligns with the general learning outcomes of CS1. The review highlights that most undergraduate studies employ the imperative-first pedagogy, offering reassurance to educators that the prevalent trend towards OOP may not be essential for teaching novice programmers. The crucial skills for beginners in programming involve understanding problem-solving techniques and implementing solutions effectively. Additionally, the review mentions a preference for block-based programming in K-12 education, but emphasizes that its main focus remains on courses offered to novice undergraduate students.

Active learning in programming education has been systematically analyzed. [Calderon et al. \(2023\)](#) explored the extant research on active learning strategies within programming education between 2000 and 2022. This review emphasizes the active engagement of students through a variety of methodologies, strategies, approaches, and student-centric pedagogical techniques, thereby facilitating their

involvement in both teaching and learning processes where the objective is for students to utilize their knowledge in meaningful ways, leveraging higher-order cognitive skills and engaging in reflective practices to construct new knowledge. The article provides a thorough and comprehensive analysis of the active learning methodologies utilized in the instruction of programming, encompassing technologies, programming languages, and the metrics employed to evaluate student learning within the context of programming. Finally, the article proposes the development of an educational tool named **CollabProg** which is envisioned as a collaborative repository, primarily aimed at assisting instructors in integrating active learning methodologies (ALMs) into their programming curriculum. This tool is designed to help instructors identify and select ALMs that align with their specific pedagogical requirements and teaching contexts. Furthermore, CollabProg will offer a comprehensive set of guidelines detailing the procedural steps for instructors to effectively implement these methodologies in their classroom settings. This resource seeks to streamline the adoption of active learning by providing structured support and fostering a community of practice among educators.

Despite the breadth of existing literature, research gaps remain in undergraduate CSE with respect to AI integration. Recent reviews by [Gaitantzi and Kazanidis \(2025\)](#), [Zhu and Zhang \(2025\)](#) have addressed overlapping themes related to the use of AI in STEM and computer science education, our work offers a distinct and complementary contribution. [Zhu and Zhang \(2025\)](#) present a broader perspective

on AI in Higher degree CSE, without a focused analysis of undergraduate CSE curricula or technical AI classifications. Gaitantzi and Kazanidis (2025), on the other hand, provide a more narrowly scoped review centered on AI applications in database instruction, limiting their generalizability across the wider CSE domain. In contrast, **our review addresses these gaps by offering a domain-specific synthesis of AI in undergraduate computer science education.** We provide a fine-grained categorization of course types (e.g., programming, hardware, theoretical CS, and information systems), a detailed taxonomy of data sources (e.g., log files, GPA, code submissions, and demographic data), and a comprehensive mapping of AI methods (e.g., supervised learning, ensemble techniques, NLP, and deep learning). These elements, combined with our rigorous methodological approach, offer a more technically detailed and pedagogically relevant analysis of how AI is shaping educational experiences in the context of undergraduate CSE.

3. Methodological framework

This systematic literature review (SLR) adheres to the established guidelines for transparent, reproducible, and rigorous research in educational technology and computer science proposed by Kitchenham et al. (2007). This protocol was selected because this is specifically designed for software engineering SLRs. The comparison of our methodology with other well-known methodology namely PRISMA (Page et al., 2021) can be found in table Table A.2 in Appendix. Our methodology implements:

- A predefined search strategy
- Explicit inclusion and exclusion criteria
- Multi-stage screening with independent reviewers
- Formal quality assessment

This structured approach minimizes researcher bias and enhances reproducibility (Kitchenham et al., 2007). Beyond mitigating bias, SLRs synthesize evidence on phenomena across diverse contexts and methodologies. When included studies demonstrate consistent findings, they provide robust validation of the phenomenon's resilience and applicability (Sharma & Giannakos, 2020).

3.1. Search strategy

We restricted inclusion to peer-reviewed empirical studies to ensure methodological quality and focus on primary research. This approach prioritizes scholarly rigor while aligning with standard SLR practices in computer science education research. Database accessibility constraints limited initial retrieval to Google Scholar, though subsequent validation searches in the ACM Digital Library and IEEE Xplore confirmed comprehensive coverage (no additional papers found in July 2025 verification). We limited the search to peer-reviewed empirical studies published in English from 2011 onward, corresponding with the launch of the first International Conference on Learning Analytics & Knowledge (LAK), which marked a turning point for this field.

3.2. Search query construction

The search strategy for this review focused on key concepts, including 'computer science education', 'AI' and 'undergraduate'. These concepts were chosen to ensure broad coverage of research at the intersection of AI and computer science education for undergraduate students. To maximize recall and account for terminological variation in the field, the search query included not only "artificial intelligence" but also broader umbrella terms such as "Learning Analytics" (LA) and "Educational Data Mining" (EDM).

This decision was informed by prior work noting that researchers frequently report AI methods under the broader categories of LA and EDM, even when these techniques are central to their study (Bhosale

& Sawane, 2021). Predictive learning analytics, for instance, involves the application of artificial intelligence algorithms to forecast learning outcomes and tailor interventions (Tan et al., 2024).

However, as noted in multiple reviews of the field, a large portion of LA-tagged literature relies primarily on traditional statistical analysis or non-AI techniques, such as social network analysis and lag sequential analysis (Khalil & Ebner, 2016; Leitner et al., 2017).

In this review, we make an attempt to determine those artificial intelligence techniques which are being integrated in these interdisciplinary fields. Therefore, we emphasize that learning analytics and educational data mining were used strictly as search terms, and not as inclusion criteria representing the methodological focus of this review.

Hence, the following search query was applied using Boolean operators:

("computer science education") AND ("AI") AND ("educational data mining" or "learning analytics") AND ("undergraduate")

Given the broad retrieval strategy, a rigorous screening protocol was used to ensure methodological consistency. Specifically, only empirical studies applying AI techniques such as supervised learning, deep learning, natural language processing (NLP), or clustering were retained for full review. Studies that merely applied traditional statistical analyses, descriptive data mining, or visual analytics without any AI-based method were systematically excluded during the full-text screening phase.

3.3. Execution of a systematic review

The initial search was carried out in **November 2023** using **Google Scholar**, applying filters for peer-reviewed literature published after 2011. This yielded **364 articles**. After removing duplicates in addition to title and abstract screening, **189 papers** remained for further screening.

Then, authors screened all titles, abstracts, and full texts using clear inclusion and exclusion criteria explained in Table 2. Articles were included if they:

- (1) Focused on undergraduate computer science education.
- (2) Applied the AI approach.
- (3) Were peer-reviewed empirical studies written in English

Importantly, in many cases the presence of AI methods was not explicitly stated in titles or abstracts. Therefore, we carefully reviewed the methodology sections of candidate papers to determine whether AI techniques (e.g., supervised learning, clustering, deep learning, NLP) were used.

This manual verification allowed us to accurately include relevant studies that would otherwise be missed due to inconsistent terminology. After the initial screening, each of the remaining studies was assessed against eight quality assurance criteria (Table 3), evaluating aspects such as methodological rigor, credibility, relevance, and research design clarity. This resulted in a final selection of **40 high-quality papers**. To confirm the completeness of the search, we conducted an **additional search in July 2025** in the **ACM Digital Library** and **IEEE Xplore**, using the same search string and filters. This additional search did not yield any new relevant studies, suggesting the original search strategy was sufficiently comprehensive. The full search and screening process is illustrated in Fig. 1 and Table 4.

3.3.1. Inclusion and exclusion criteria

The selection phase is critical for establishing the overall validity of a literature review; therefore, it is crucial to set clear inclusion and exclusion criteria. To achieve this objective, we set language, length, venue, and duplication as the main exclusion metrics. As a result, articles which were not written in English, long articles such as Master or PhD theses or books, and not-peer reviewed articles in well-known venues were excluded from the list of selected publications.

Table 2
Inclusion and exclusion criteria for systematic review.

Inclusion criteria	Exclusion criteria
(1) Focused on computer science education	(1) Not in English
(2) AI included	(2) Master or PhD thesis or books
(3) Undergraduate level	(3) Not peer reviewed
(4) Empirical studies	(4) Duplicate articles

Table 3
Quality appraisal framework.

	Quality criteria
C1.	Is the research problem clearly addressed in the study?
C2.	Are the research objectives clearly articulated in the study?
C3.	Is the context in which the research was conducted described in sufficient detail?
C4.	Was the research design suitable for addressing the aims of the study?
C5.	Does the study clearly define the research methods, including subjects, instruments, data collection, and data analysis?
C6.	Were rigorous data analysis techniques employed in the study?
C7.	Are the research findings presented clearly?
C8.	Does the study offer valuable contributions to research or practical applications?

Inclusion Criteria was mostly recognized and applied to filter the selection process during the initial scanning of the articles to ensure that they fit within the scope of computer science education, an artificial intelligence approach is applied in the article, the article is focused on the students in undergraduate level, and it is categorized as an empirical research paper. A short summary of the inclusion and exclusion metrics can be found in [Table 2](#).

3.3.2. Quality assurance framework

To ensure methodological robustness, we adopted a structured quality appraisal framework comprising eight predefined criteria (C1–C8), grounded in established systematic review guidelines for empirical software engineering and educational technology research ([Dybå & Dingsøyr, 2008](#); [Mangaroska & Giannakos, 2018](#)). These criteria collectively address three essential dimensions of study quality:

- (1) **Rigor:** Appropriateness of research methodologies.
- (2) **Credibility:** Reliability and validity of findings.
- (3) **Relevance:** Significance to educational and computer science education communities.

This framework enabled rigorous screening of studies, ensuring only high-quality, methodologically sound research informed our synthesis. Given the critical role of selection validity in systematic reviews, we applied these criteria consistently during the inclusion phase. [Table 3](#) details the eight operationalized metrics, implemented using the protocol previously followed by [Sharma et al. \(2024\)](#) to ensure transparent and replicable quality assessment. The example of how the (C1–C8) quality metrics have been applied by the authors on [Kaur and Chahal \(2023\)](#) as one of the selected papers can be found in [Table A.1](#) in [Appendix](#).

3.3.3. Data extraction and thematic coding

All the thematic coding in the paper followed the same deductive coding process. Both authors read and coded the data extracted from the selected studies for the various aspects, for example, learning behavior, learning performance, and outcomes. Then, the following scheme was followed:

Table 4
Systematic literature review screening process.

Phase	Input	Output	Criteria
Initial	364 records	189 papers	Deduplication + title/abstract screening
Eligibility	189 papers	44 papers	Inclusion or Exclusion criteria (Table 2)
Quality	44 papers	40 papers	Quality framework (Table 3)

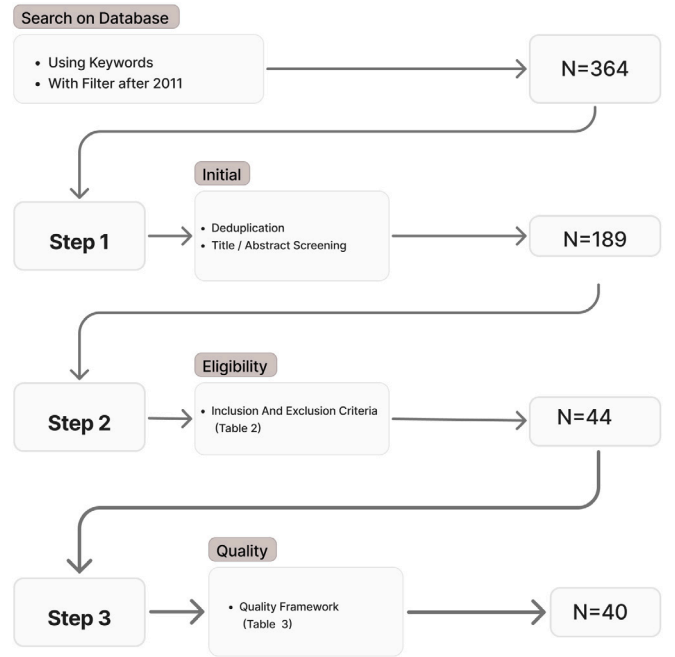


Fig. 1. Systematic literature review screening process.

- (1) 10% of the papers were coded together to establish the coding scheme.
- (2) Another 10% of papers were coded separately by the two authors, and then the codes were discussed and refined, if required.
- (3) Another 30% of the papers were coded separately by the two authors, and then the assigned codes were discussed. At this stage, the **Cohen's Kappa (κ)** ([Cohen, 1960](#)) was 0.92. This showed an alignment of understanding of the two authors about the coding scheme.
- (4) Finally, the two authors coded 50% of the papers separately.

4. Results & findings

In this section, the answer to the previously defined research questions and a short discussion of the most significant findings and results are presented and discussed in detail.

4.1. RQ1: What artificial intelligence methods and data types have been reported in undergraduate CSE research?

To answer RQ1, we examined two central dimensions of AI implementation in undergraduate computer science education: the characteristics of AI models and the types of data leveraged to support them. where each of the aforementioned perspectives are explored in detail in the subsequent paragraphs.

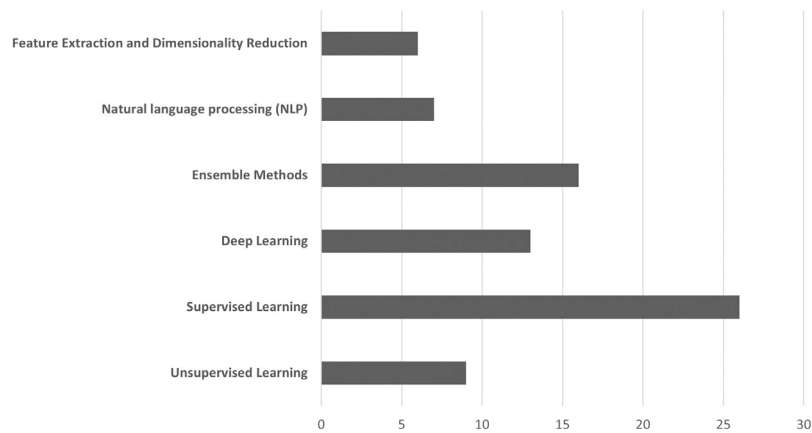


Fig. 2. The AI categories.

4.1.1. RQ1a: Which artificial intelligence methods have been applied in the state-of-the-art?

We have divided the methods used in the selected publications into six main categories namely: Feature extraction & dimensionality reduction, Natural language processing (NLP), Ensemble methods, Deep Learning, Supervised, and Unsupervised learning. Other techniques such as: Reinforcement Learning, and Explainable AI were also recognized in the state-of-the-art. However, these methods have been reported to be applied in less than three publications. Therefore, we decided to evade listing them within the most significant categories which have been in use. The distribution of the methods is depicted in Fig. 2 where Supervised Learning, Ensemble Methods, and Deep Learning are among the most popular methods within the domain.

4.1.2. RQ1b: Which type of data have been collected in this field?

The collected data in the state-of-the-art varies to a wide extend. To be able to have a better understanding of the nature of this data, we have divided it into five main categories namely: Log and trace data, CGPA and academic performance, questionnaires, demographic and personal information, and programming & code-related data. Fig. 3 depicts the data distribution where “Log and trace data” followed by “CPGA and academic performance” and “Programming and code-related data” have been reported among the most popular data types.

- **Log and trace data:** This category includes log data from students’ interactions with Moodle, quiz module, and log files of students’ interaction with robots. It also includes log data from Google Forms surveys, as well as log data from Blackboard LMS analytic.
- **CGPA and academic performance:** This category includes CGPA from previous semesters, grades obtained from the first two to eight weeks of the semester, course-level grades, and GPA. It also includes academic performance data from pre-test and post-test results.
- **Questionnaires:** This category includes questionnaires and surveys.
- **Demographic and personal information:** This category includes demographic data such as gender, age, race, work, family, distance from university, IP address, and student name.
- **Programming and code-related data:** This category includes programming problems in online judge, code snippets from online tutorials, code bench dataset for recommendation part, students’ code submissions, and meta data such as student ID, assignment name, and time stamp.

4.1.3. Three case studies

Goundar et al. (2022) used Decision Tree, Random Forest, Naïve Bayes and Support Vector Machine on the Demographic and Academic data from Moodle (LMS) to see the possibility of predicting data from Moodle logs and demographic data provided by the Student Academic Services. The results indicate that student interactions have a direct impact on student retention. Students that interact more perform better and they continue with their studies, while those that do not interact much, fail and drop out.

Smirani et al. (2022a) used three ensemble learning classifiers—Light Gradient Boosting Machine (LGBM), eXtreme, Gradient Boosting machine (XGB), and Random Forest (RF), with a Multilayer Perceptron (MLP) meta-classifier on the Data collected from blackboard Learning Management analytic to see the possibility of predicting data from Moodle logs and demographic data provided by the Student Academic Services. The results indicate that findings showed a significant increase in student success rates (98.86%), and a drop rate from 12% to 1.14%.

Qushem et al. (2024) used Decision Trees, Random Forest, Naïve Bayes, K-Nearest Neighbor, Neural Networks, and Support Vector Machine, Boruta algorithm on the course-level grades and GPA collected from the Learning Management analytic to study underscore the importance of course grades in predicting students’ overall academic performance. The results suggested that early identification of at-risk students based on their performance in key courses could enable targeted interventions to improve their academic outcomes

4.2. RQ2: How is the educational context in terms of targeted courses and the technologies employed?

We have evaluated the educational context based on the type of courses targeted in the studies, and the technologies in used within course development or experimental phases.

4.2.1. RQ2a: which courses have been targeted?

Our findings demonstrate that 75 percent of the articles involved in this literature review are focused on the experiments conducted on the programming education. Fig. 4 demonstrates the distribution of courses being taught in targeted publications where 30 out of 40 research papers were solely focused on programming. Therefore, the findings in this research are aligned with the results in Apiola et al. (2023) which reported the significant research focus on programming education within the field of study. However, our review also highlights a *broader array of course contexts* beyond programming. To better characterize this diversity, we classified the educational settings into more fine-grained categories:

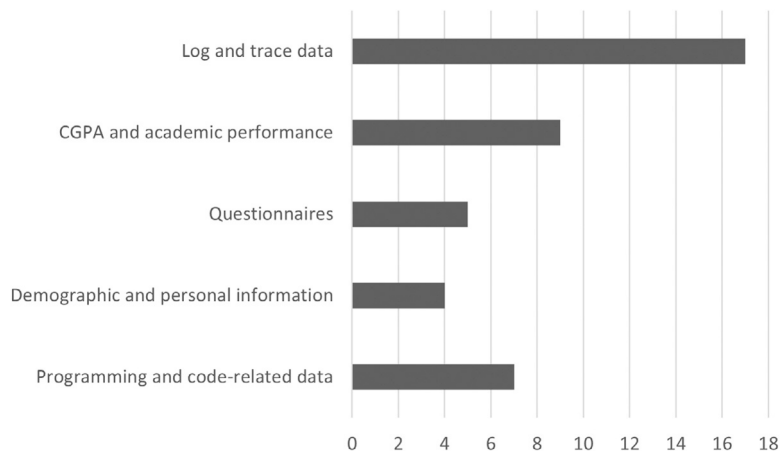


Fig. 3. The collected data categories.

- **Mathematics and Quantitative Reasoning Courses:** *Discrete Mathematics* (Maniktala et al., 2020); *Linear Algebra II*, *Probability III*, *Optimization Theory I*, *Mathematical Modeling* (Qushem et al., 2024).
- **Hardware and Architecture Courses:** *Microprocessors* (Asthana et al., 2023); *Computer Architecture II*, *Basic Hardware Maintenance* (Qushem et al., 2024).
- **Systems and Theoretical Computing Courses:** *Operating Systems II*, *Compiler Construction I*, *Design and Analysis of Algorithms*, *Computer System Modeling & Evaluation*, *Computer Graphics* (Qushem et al., 2024).
- **Applied and Information Systems Courses:** *Database Design and Management*, *System Analysis and Design*, *Website Development*, *Networks*, *Information Technology*, *Programming Python 1*, *Analysis and Design of Information Systems* (Qushem et al., 2024; Smirani et al., 2022b).
- **Interdisciplinary and Methodological Courses:** *Entrepreneurial Development*, *Research Methodology*, *Operations Research* (Qushem et al., 2024).
- **Unspecified or Composite Curricula:** A number of studies examined students enrolled in broader computing degree programs or grouped data across multiple courses. For example, Atalla et al. (2023) proposed a curriculum-aware recommendation system for academic advising within a BSc in Computing, encompassing both programming and non-programming courses. Similarly, Qushem et al. (2024) analyzed datasets comprising both core CS courses (e.g., CS201, CS304) and general education or management courses required for degree completion. Tian et al. (2021) studied an introductory computer science course, though the course's exact nature is not specified. Messaoudi et al. (2022) focused on a course titled *Computer Science 3*, without further detail. Naseem et al. (2022) analyzed freshmen attrition in a general *Computing Science* program.

Overall, while programming remains the most studied instructional setting, the presence of AI-enhanced prediction and learning tools in diverse subjects ranging from theoretical computer science and hardware to applied systems and interdisciplinary topics demonstrates the growing breadth of AI applications in computing education. All studies reviewed involved students enrolled in computer science or closely related undergraduate degree programs.

4.2.2. RQ2b: which technologies have been used?

We have divided the technologies discovered in the state-of-the-art into four main categories, namely learning management systems,

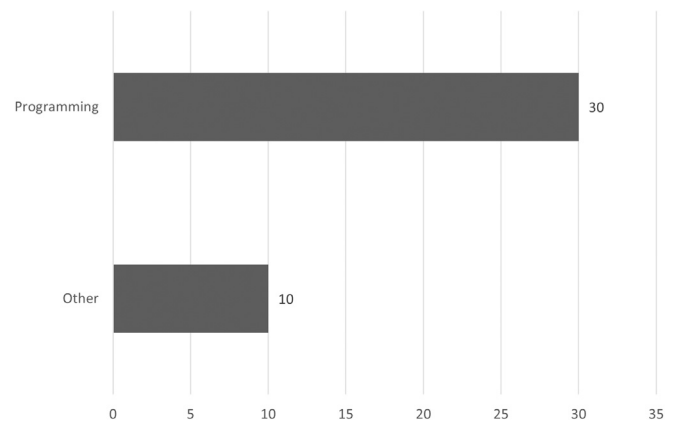


Fig. 4. Course categories.

programming and assessment tools, development environments and tools, and game-based learning environments.

- **Learning Management Systems (LMS):** Platforms like Moodle (de Melo-Minardi et al., 2022) and VILLE (Veerasamy et al., 2022) stand out as comprehensive LMS solutions, facilitating course management, content delivery, and assessment, thereby serving as the backbone of online education by providing a centralized platform for both instructors and students.
- **Programming and Assessment Tools:** Tools like Codebench (Pereira et al., 2023), AAT-Moodle (López-Pernas & Saqr, 2021) for programming automated assessment, and various online judges (e.g., Javaluador (Rico-Juan et al., 2023), self-developed SQL online Judge (Xu et al., 2022)) are pivotal in the computer science education sector. They automate the evaluation of code submissions, enhancing the learning process through immediate feedback and efficiency.
- **Development Environments and Tools:** Snap (Gao et al., 2021) and Alice 3 (Yusuf et al., 2023) offer block-based programming environments that simplify coding for beginners, while Git (Karakaš & Helic, 2023) provides a version control system essential for collaborative development projects. The use of Lego Mindstorms EV3 kits (Scaradozzi et al., 2021) for robotics introduces students to programming through hands-on learning.

Table 5

The discovered themes and the subsets of papers within.

Learning behavior
1. Educational Engagement and Understanding
2. Behavioral Patterns and Learning Outcomes
3. Tailored Instructional Strategies
4. Predictive Modeling and Early Intervention
5. Technology and Learning Analytics
6. Adaptive Learning Environments
Learning performance and outcomes
1. Predictive Analytics for Student Performance
2. Impact of Behavioral Profiles on Academic Achievement
3. Tailored Educational Interventions based on Predictive Models
4. Correlation between Behavioral Patterns and Learning Outcomes
5. Socio-economic Factors and Learning Outcomes
6. Equitable Skill Development through Data-Driven Methods
At risk learner
1. Early Identification of At-Risk Learners
2. Predictive Models for Academic Performance
3. Tailored Educational Interventions based on Predictive Insights
4. Proactive Approach to Addressing Academic Challenges
Interventions
1. Design and Implementation of Targeted Educational Interventions
2. Technological Interventions for Academic Support
3. Development of Predictive Tools for Academic Performance
Hints or recommendations
1. Automated Problem Selection in Programming Education
2. Personalized Learning Support through Recommender Systems
3. Enhancing Learning with Intelligent Hint Delivery Mechanisms
Automatic content extraction
1. Automatic Content Extraction for Learning Material Enhancement
2. Automated Classification and Categorization of Educational Content

- **Game-based Learning Environments:** Technologies such as the Escapp platform (López-Pernas et al., 2023) and Unity 3D (Lin & Chen, 2020) for Augmented Reality (AR) development offer students immersive and interactive learning experiences, leveraging game-based learning and AR to facilitate engagement and deeper understanding of complex subjects.

4.2.3. Three case studies

Manikitala et al. (2020) used log data, from an Intelligent Tutoring System (Deep thought as an intelligent tutoring which uses Hint Factory), in a discrete mathematics class to propose and evaluate a new hint delivery mechanism called “Assertions” for providing unsolicited hints in a data-driven intelligent tutor.

Carmen (2022) used data from an LMS in a computer architecture course to show that integrating online training activities into the learning process improves students’ results, especially in the short term. Moreover, the parameter that most influences the cognitive outcomes is the number of times a training activity is completed.

Hawlitschek et al. (2020) used data from a programming platform used in an introductory programming course to show that early dropouts could be predicted by the delay between assignment availability and the first assignment submission, and that correlated with prior programming experience and job hours. For later dropouts, the number of errors in the first assignment was a key predictor.

4.3. RQ3: What are the research objectives and findings in the current state-of-the-art?

To highlight the research objectives and outcomes from the 40 contributions included in the SLR, we conducted a second-level coding that produced six themes where the overview of each theme and its subcategories can also be found in Table 5. In the following paragraphs, we will discuss each theme in detail.

4.3.1. Learning behavior

This section looks into various aspects of student engagement and behavior within educational settings. It is divided into six sub-themes, each exploring different dimensions of learning behavior.

1. Educational Engagement and Understanding: This set of papers focuses on how students engage with educational materials and platforms, such as learning management systems (e.g., Moodle), and how this engagement correlates with their understanding of the subject matter. Studies within this theme investigate various forms of engagement, such as frequency of logins, participation in discussions, and interaction with course materials. The goal is to understand how active participation and consistent engagement contribute to enhanced learning outcomes and improved understanding of complex topics (Carmen, 2022; Goundar et al., 2022; Tian et al., 2021).

2. Behavioral Patterns and Learning Outcomes: The second set of papers in this theme, delves into the behavioral patterns exhibited by students in educational settings and examines how these patterns impact their learning outcomes. Studies analyze behaviors such as persistence, procrastination, engagement levels, and interaction with course materials. By identifying and understanding these patterns, researchers aim to correlate them with academic performance, dropout rates, and overall success in learning. The goal is to uncover insights into how specific behaviors contribute to or hinder learning outcomes (Blikstein et al., 2014; Lokkila et al., 2022; López-Pernas et al., 2023; Messaoudi et al., 2022; Moraes & Pedreira, 2021; Fahid et al., 2021; Real et al., 2020; Rico-Juan et al., 2023; Scaradozzi et al., 2021).

3. Tailored Instructional Strategies: Within this set of contributions, researchers explore the development of instructional strategies tailored to individual student needs and preferences. Studies aim to categorize students into different groups based on their behavioral profiles and learning strategies. By understanding these profiles, educators can design interventions that cater specifically to the needs of diverse learner types. The goal is to optimize teaching methods and educational interventions to enhance student engagement, understanding, and academic success (Kaur & Chahal, 2023; López-Pernas et al., 2023; Tian et al., 2021).

4. Predictive Modeling and Early Intervention: The fourth set of papers focuses on the use of predictive modeling techniques to identify early indicators of student success or risk of failure. Researchers analyze historical data and behavioral patterns to predict student outcomes and intervene proactively to support struggling students. By understanding how specific behaviors correlate with learning outcomes, educators can implement targeted interventions to improve overall student success rates and reduce dropout rates (Hawlitschek et al., 2020; Messaoudi et al., 2022; Real et al., 2020; Smirani et al., 2022b).

5. Technology and Learning Analytics: Within this set of contributions, researchers leverage technology and learning analytics techniques to analyze student behavior and interactions with educational platforms and materials. Studies explore how students engage with learning management systems, online activities, and educational tools. By analyzing this data, researchers aim to extract insights into student behavior, preferences, and learning strategies. The goal is to inform instructional design and decision-making in education by understanding how technology can enhance the learning experience and improve educational outcomes (Goundar et al., 2022; Real et al., 2020; Rico-Juan et al., 2023; Scaradozzi et al., 2021).

6. Adaptive Learning Environments: Finally, the last set of papers examines the design and implementation of adaptive learning environments that cater to diverse learner needs and preferences. Researchers explore how students engage with different learning environments, such as traditional classrooms, online platforms, and educational activities. By understanding student behavior and learning strategies, educators can design adaptive learning environments that personalize the learning experience and optimize student engagement and understanding. The goal is to create flexible and responsive learning environments that support individualized learning pathways and enhance overall learning outcomes (Kaur & Chahal, 2023; López-Pernas et al., 2023; Scaradozzi et al., 2021).

4.3.2. Learning performance and outcomes

This section examines the various factors that influence student performance and learning outcomes in educational settings. It covers six sub-themes, each focusing on how data and behavioral analysis can enhance educational strategies and student success.

1. Predictive Analytics for Student Performance: The first set revolves around the use of predictive analytics and machine learning techniques to forecast student performance in educational settings. Studies within this theme analyze various types of data, including demographic information, learning behaviors, and engagement metrics, to develop predictive models. These models aim to identify students who may be at risk of underperforming early in the semester. By leveraging predictive analytics, educators can implement timely interventions to support struggling students and enhance overall student outcomes (Asthana et al., 2023; Buschetto Macarini et al., 2019; Chung & Hsiao, 2020; de Melo-Minardi et al., 2022; Goundar et al., 2022; Karakaš & Helic, 2023; Maier et al., 2021; Smirani et al., 2022b).

2. Impact of Behavioral Profiles on Academic Achievement: Within this set of contributions, researchers investigate how specific behavioral profiles and learning behaviors correlate with academic achievement. Studies explore behaviors such as engagement levels, persistence, gaze direction, and disruptive episodes during learning sessions. The goal is to understand how these behaviors influence students' academic performance and success. By identifying the impact of behavioral profiles on academic achievement, educators can develop strategies to promote more engaged and productive learning behaviors among students (Blikstein et al., 2014; Lokkila et al., 2022; Maniktala et al., 2020; Fahid et al., 2021; Silveira et al., 2022; Veerasamy et al., 2022; Yusuf et al., 2023).

3. Tailored Educational Interventions based on Predictive Models: The next set focuses on the development and implementation of tailored educational interventions based on predictive models of student performance. Researchers analyze predictive models to classify students based on their likelihood of underperforming. These classifications enable educators to implement targeted interventions, such as customized learning paths, supplementary materials, and additional support resources, to improve student outcomes. By tailoring interventions to individual student needs, educators can optimize learning experiences and enhance student success (Asthana et al., 2023; Buschetto Macarini et al., 2019; Smirani et al., 2022b).

4. Correlation between Behavioral Patterns and Learning Outcomes: This set explores the correlation between specific behavioral patterns exhibited by students and their learning outcomes. Researchers examine behaviors such as engagement with course materials, persistence in completing tasks, and responsiveness to feedback. The goal is to identify behavior patterns that predict learning success and academic achievement. By understanding how behavioral patterns impact learning outcomes, educators can develop strategies to support more effective learning behaviors and improve overall student performance (Blikstein et al., 2014; Karakaš & Helic, 2023; Lokkila et al., 2022; Maniktala et al., 2020; Fahid et al., 2021; Veerasamy et al., 2022; Yusuf et al., 2023).

5. Socio-economic Factors and Learning Outcomes: There was one paper in this theme where researchers investigate how socio-economic factors influence student success rates in academic courses, particularly in computing science education. Studies explore the relationship between socio-economic backgrounds and failure rates in specific majors or courses. The goal is to understand how socio-economic factors shape learning outcomes and identify strategies to support students from diverse socio-economic backgrounds (Silveira et al., 2022).

6. Equitable Skill Development through Data-Driven Methods: Finally, the last set of papers focuses on leveraging data-driven methods to promote equitable skill development among students. Researchers utilize techniques such as machine learning, data analysis, and educational modeling to optimize learning experiences and ensure equitable

outcomes for all students. Studies within this theme explore how data-driven methods can level the playing field, reduce initial advantages held by certain students, and promote skill development among diverse student populations. By leveraging data-driven methods, educators can design more inclusive and effective learning environments that support equitable skill development for all students (Albreiki et al., 2023; Lin & Chen, 2020; Lokkila et al., 2022; López-Pernas & Saqr, 2021).

These papers encompass various aspects of understanding student behavior, engagement, and learning outcomes in educational contexts, particularly within the field of computing science education. They highlight the importance of leveraging data-driven approaches and predictive analytics to support student success and promote more effective teaching and learning strategies.

4.3.3. At-risk learner

This section explores strategies for identifying and supporting students who are at risk of poor academic outcomes. It focuses on the use of data analysis and predictive modeling to provide timely interventions and improve student retention and success.

1. Early Identification of At-Risk Learners: The first set of papers, focuses on research aimed at identifying students who are at risk of poor academic outcomes early in their academic journey. Studies within this theme utilize various data analysis techniques, such as predictive analytics and machine learning algorithms, to detect subtle patterns in student behavior, performance, and engagement. The goal is to proactively identify students who may be at risk of low grades, course failure, or dropout and implement timely interventions to support them before their academic difficulties escalate. Early identification of at-risk learners enables educators to provide targeted assistance and resources, thereby increasing the likelihood of academic success and retention (de Melo-Minardi et al., 2022; Hawlitschek et al., 2020; Veerasamy et al., 2022).

2. Predictive Models for Academic Performance: Within this set, researchers develop predictive models using advanced data analysis methods to forecast student performance and identify students at risk of failing or dropping out of their courses. These models leverage a variety of data sources, including demographic information, academic records, and engagement metrics, to predict the likelihood of student attrition. By analyzing patterns and trends in student data, predictive models can provide valuable insights into factors that contribute to academic success or failure. Educators can use these insights to intervene proactively and provide personalized support to at-risk students, ultimately improving their academic outcomes and retention rates (Buschetto Macarini et al., 2019; Naseem et al., 2022; Qushem et al., 2024).

3. Tailored Educational Interventions based on Predictive Insights: These papers emphasize the design and implementation of targeted educational interventions based on predictive insights derived from data analysis. Researchers develop interventions, such as additional tutoring, personalized support, and adjustments in teaching strategies, to address the specific needs of at-risk students identified through predictive modeling. By tailoring interventions to individual student needs, educators can provide timely and effective support that enhances student engagement, motivation, and academic performance. The goal is to leverage predictive insights to inform decision-making and improve outcomes for at-risk learners (Buschetto Macarini et al., 2019; Carneiro et al., 2022).

4. Proactive Approach to Addressing Academic Challenges: Studies within this set adopt a proactive approach to addressing academic challenges by identifying at-risk learners early in their academic journey. Researchers analyze early engagement and performance data to predict potential dropout risks and detect subtle differences in student interactions with course materials. By forecasting potential dropouts early on, educators can implement targeted interventions and support strategies to increase retention rates and improve educational outcomes. This proactive approach ensures that at-risk students receive timely assistance and resources to help them succeed academically (de

Melo-Minardi et al., 2022; Veerasamy et al., 2022). These papers underscore the importance of leveraging data-driven approaches and predictive analytics to identify and support at-risk learners in academic settings. By identifying early warning signs of academic difficulty and implementing targeted interventions, educators can improve student outcomes, retention rates, and overall academic success.

4.3.4. Interventions

This section explores the design and implementation of various interventions aimed at improving student outcomes, particularly for those at risk of academic struggles. It focuses on the use of predictive analytics and technology to provide targeted support and enhance learning experiences.

1. Design and Implementation of Targeted Educational Interventions: This paper focuses on research that emphasizes designing and implementing specific educational interventions based on predictive insights derived from data mining or analytics. The goal is to accurately identify students who are at risk of academic struggles, such as failing courses, and then deploy targeted interventions to address their specific needs. These interventions may include additional tutoring, personalized support, or adjustments in teaching strategies tailored to the identified challenges of at-risk students. By intervening early and addressing potential obstacles to learning, educators aim to improve student outcomes and increase overall academic success rates (Carneiro et al., 2022).

2. Technological Interventions for Academic Support: Studies within this set explore the use of technological interventions to provide academic support and enhance learning outcomes. These interventions often involve the implementation of intelligent tutoring systems, chatbots, or automated feedback mechanisms to deliver personalized assistance and guidance to students. The focus is on leveraging technology to foster positive behavioral changes, clarify misunderstandings, reinforce correct understanding, and encourage consistent study habits, particularly among students who may be at risk of underperformance. By integrating technology into educational practices, educators aim to enhance the effectiveness of support strategies and improve student engagement and retention (Scaradozzi et al., 2021; Sychev et al., 2021; Tian et al., 2021).

3. Development of Predictive Tools for Academic Performance: This category revolves around the development of predictive tools or systems that utilize machine learning algorithms to forecast student performance based on their academic profiles. The primary objective is to provide academic advisors and students with actionable insights into course readiness and necessary preparations. By predicting student performance, these tools enable proactive interventions and support strategies to address potential learning difficulties before they escalate. Additionally, these predictive tools empower educators to monitor student progress in real-time and make data-driven decisions to optimize instructional strategies and enhance student outcomes (Messaoudi et al., 2022).

The aforementioned themes collectively underscore the importance of leveraging technology and predictive analytics to support student learning and improve academic success rates. By implementing targeted interventions, leveraging technological innovations, and developing predictive tools, educators can better identify and address the needs of at-risk students, ultimately fostering a more inclusive and supportive learning environment.

4.3.5. Hints or recommendations

This section explores the development and implementation of recommender systems and intelligent mechanisms designed to provide personalized support and enhance the learning experience. The focus is on leveraging technology and data-driven approaches to optimize educational outcomes.

1. Automated Problem Selection in Programming Education: This set of papers revolves around the development and implementation

of recommender systems that automate the process of selecting programming problems for students. Instructors face challenges in curating a diverse set of problems that align with the curriculum's objectives, difficulty levels, and student needs. Automated systems, such as the ones described in the categorized studies, leverage data-driven analysis and problem descriptions to recommend appropriate problems to students. By automating this process, instructors can save time and effort in selecting problems, ensuring that students are presented with relevant and engaging challenges that contribute to their learning progress (Pereira et al., 2023, 2022).

2. Personalized Learning Support through Recommender Systems: In this paper, the focus is on providing personalized learning support to students through the use of recommender systems. These systems analyze vast amounts of data, including student records and curriculum information, to identify individual learning needs and preferences. By leveraging big data and artificial intelligence, personalized learning paths and recommendations are generated, guiding students through their educational journey. The goal is to enhance the effectiveness of academic advising and support by tailoring learning experiences to each student's unique background, interests, and academic goals (Atalla et al., 2023).

3. Enhancing Learning with Intelligent Hint Delivery Mechanisms: These two papers explore innovative approaches to delivering hints and recommendations within educational environments. Intelligent hint delivery mechanisms, such as Assertions and deep learning recommendations, are designed to provide contextually relevant and engaging support to students. By seamlessly integrating with learning platforms and environments, these mechanisms aim to enhance student engagement, problem-solving skills, and overall learning outcomes. The studies within this theme demonstrate the potential of technology-driven hint delivery to optimize the learning experience and empower students to overcome challenges more effectively (Lin & Chen, 2020; Maniktala et al., 2020).

These themes collectively highlight the importance of leveraging technology, data analysis, and artificial intelligence to enhance various aspects of programming education, including problem selection, personalized support, and intelligent hint delivery. By incorporating these innovative approaches into educational practices, institutions can create more engaging, effective, and tailored learning experiences for students.

4.3.6. Automatic content extraction

This section examines the use of technologies and algorithms to automate the extraction and categorization of educational content. The goal is to enhance learning materials and create personalized learning experiences.

1. Automatic Content Extraction for Learning Material Enhancement: These three papers revolve around the application of technologies and algorithms to automatically identify and extract relevant information from digital educational content. The primary goal is to enhance learning materials by simplifying content delivery, adapting content based on extracted information, and creating more personalized learning experiences. Studies falling under this theme utilize machine learning models, convolutional neural networks (CNNs), natural language processing (NLP) techniques, and semantic models to automatically analyze and categorize educational content. (de Melo-Minardi et al., 2022; Moraes & Pedreira, 2021; Pereira et al., 2023).

2. Automated Classification and Categorization of Educational Content: This paper focuses specifically on the automated classification and categorization of educational content, particularly programming problems. By leveraging advanced NLP techniques and semantic models, these studies aim to automatically classify programming problems into relevant academic disciplines or syllabus topics. The ultimate goal is to tailor programming assignments to enhance their educational value for students from diverse backgrounds (Fonseca et al., 2020).

These themes highlight the significance of automatic content extraction techniques in improving educational content delivery, organization, and accessibility, ultimately enhancing the learning experience for students.

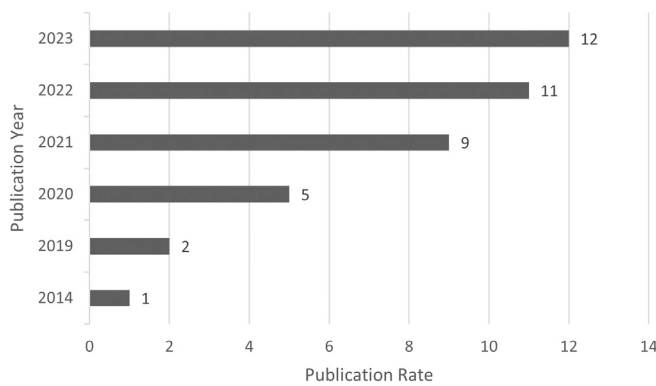


Fig. 5. The Covid impact on publication rate.

4.4. RQ4: Are there any notable shifts in publishing trends before and after the COVID-19 pandemic?

State-of-the-art demonstrates that the pandemic has significantly accelerated the adoption of AI-based approaches in education. Pilot studies indicate that there was a marked increase in the use of AI-driven e-learning platforms during this period (Pantelimon et al., 2021). This shift reflects a broader trend towards integrating more sophisticated technologies in educational settings to enhance learning experiences and outcomes (Mallik & Gangopadhyay, 2023).

In this research question, we made an attempt to see whether the same frequency applies to the AI-driven approaches in computer science education or not. According to our evaluation, 89 percent of papers published within the domain have been released since 2020, and after pandemic. More detail in this perspective can be found in Fig. 5.

4.5. RQ5: What are the opportunities and challenges in using AI in undergraduate CSE?

The application of AI techniques in undergraduate computer science and engineering (CSE) education presents both opportunities and challenges. One key opportunity lies in the potential of generative AI models to enhance the learning experience for CSE students. Generative AI, with its ability to create novel content, can be leveraged to generate personalized learning materials, practice problems, and even code examples tailored to individual student needs. This can lead to more engaging and effective learning, as students are exposed to a diverse range of relevant examples and exercises.

However, the limited evidence of generative AI usage in the evaluated papers suggests that this potential has not been fully realized in undergraduate CSE programs.

Additionally, the concept of explainability in AI, which is crucial for fostering student understanding and trust in the technology, has only been explored in a few studies.

5. Discussion and future work

This section is primarily divided into three parts. First is the general discussion dealing with aspects such as how AI applications may affect core educational values; certain perspectives of ethics in AI; and which methodological aspects we can bring from other fields that have more systematic approach of incorporating new AI-technologies. Second part of this discussion is about the specific challenges in CSE and use of AI. Finally, the third part of the discussion presents the opportunities of using AI in CSE.

5.1. General discussion

5.1.1. How AI applications may affect core educational values

Students and educators often do not understand how AI systems make decisions, such as predictive models and adaptive learning algorithms. Lack of transparency undermines trust in educational recommendations, interventions, and grading. AI in education needs Explainable AI (XAI) approaches to provide clear, interpretable insights into how decisions (e.g., risk predictions, learning path recommendations) are made. Transparency also includes communicating data collection practices, model limitations, and the purpose of AI tools (Chaushi et al., 2023; Khosravi et al., 2022; Marconi & Bandini, 2021; Rachha & Seyam, 2023; Raees et al., 2024; Zoppis et al., 2020).

Educational inequalities can be mitigated or exacerbated by AI. AI systems trained on biased data may unintentionally reinforce disparities, disadvantaging underrepresented students. Meanwhile, AI can enhance fairness by ensuring consistent, unbiased feedback and personalizing learning for diverse learners. A fair learning model, bias audits, and inclusive datasets based on diverse student data are needed to ensure fairness (Chaushi et al., 2023; Khosravi et al., 2022; Marconi & Bandini, 2021; Rachha & Seyam, 2023; Raees et al., 2024; Zoppis et al., 2020).

The use of AI can help promote inclusiveness by providing adaptive learning experiences tailored to the needs of individual students. Multilingual support, accessibility features, and learning resources tailored to varying skill levels can all be provided by AI. Achieving inclusiveness, however, requires a clear understanding of diverse learner contexts and equitable access to AI-driven tools (e.g., considering students with limited internet or device access).

AI will not replace teachers, but reshape their roles. In the future, educators may become facilitators, mentors, and interpreters of AI-generated insights. Teachers must learn how to integrate AI tools ethically and effectively in pedagogy. A human teacher's empathy, motivation, and ability to facilitate critical thinking remain irreplaceable, but AI can handle routine tasks like grading and feedback.

5.1.2. Perspectives from ethics in AI

In educational contexts, ethics in AI provides a framework for addressing privacy, fairness, accountability, transparency, and autonomy concerns. AI systems used in classrooms must be accountable (Memarian & Doleck, 2023). When AI systems make mistakes, e.g., misclassifying students or providing misleading feedback, clear responsibility must be assigned. AI decisions and outputs should be overseen by teachers.

Student behavior, performance, and sometimes personal background data are collected for AI applications. Data protection regulations (e.g., GDPR, FERPA) and respecting student autonomy through informed consent and data minimization are ethical requirements (Hoel et al., 2017; Huang, 2023).

AI ethics emphasize continuous bias audits. This involves systematically evaluating AI models to detect, mitigate, and correct any discriminatory patterns in how the AI interacts with different student groups (Simbeck, 2024).

An ethical AI framework demands that students and educators understand how AI-driven decisions are made. Explainability is not just a technical feature but an ethical necessity, ensuring that stakeholders can question and interpret AI recommendations (Chaushi et al., 2023; Khosravi et al., 2022; Marconi & Bandini, 2021; Rachha & Seyam, 2023; Raees et al., 2024; Zoppis et al., 2020).

AI should aim to benefit students (beneficence) and avoid causing harm (non-maleficence). This principle entails rigorous testing of AI interventions to ensure they truly support learning and do not inadvertently widen achievement gaps or reduce student agency (O'Donoghue, 2023).

5.1.3. Importing methodological insights from medical AI into education AI

One of the fields that has incorporated AI-interventions in a very systematic manner, is medical AI. In this subsection, we will briefly present which methods can we adopt from medical AI. In medical AI, clinician oversight is critical to final decisions (Herman et al., 2024). In education, AI should augment rather than replace educators, with teachers interpreting AI outputs, intervening when necessary, and providing the human judgment AI lacks.

The healthcare industry prioritizes explainable models, especially for diagnostics and treatment recommendations (Borys et al., 2023). Educators and students can understand how learning analytics or predictions are derived by using model-agnostic explanation tools (e.g., SHAP, LIME; (Chaushi et al., 2023; Khosravi et al., 2022; Marconi & Bandini, 2021; Rachha & Seyam, 2023; Raees et al., 2024; Zoppis et al., 2020).

For errors, model drift, and unintended consequences, medical AI needs rigorous validation through clinical trials (Li et al., 2022). In educational AI, real-world impact should be assessed and models updated accordingly to maintain accuracy and fairness.

To ensure equal treatment across demographics (e.g., race, gender), medical AI conducts fairness audits (van de Sande et al., 2022). In order to identify and address disparities between predictive models or adaptive systems, education AI should implement bias detection pipelines.

Education AI systems should follow structured ethical AI governance frameworks (Herman et al., 2024; Li et al., 2022), just as medical AI follows ethical guidelines (e.g., informed consent, data privacy). Involve stakeholders in AI deployment decisions, report risks transparently, and comply with privacy laws.

In medical AI, risk-benefit frameworks ensure AI interventions do not harm patients (Alhitmi et al., 2024). Similarly, AI applications in education need impact assessments to ensure interventions enhance learning while protecting students.

5.2. Challenges

In this research, we have recognized five primary challenges: challenges in computer science education, student engagement, automatic content extraction, identifying at-risk Students, implementing interventions and recommendations. Below, we provide a brief overview of each aspect.

5.2.1. Challenges in computer science education

Computer science education faces numerous challenges that impact student learning and overall educational effectiveness including complexity in assessing student performance, rapid technological changes, resource limitations, and ethical & privacy concerns.

Students' performance in computer science education presents several challenges. Due to the complexity of computer science, **assessing students' performance** can be challenging. Students' understanding of concepts, problem-solving abilities, and practical skills in coding and software development may not be adequately captured by traditional assessment methods (Paiva et al., 2022). Inadequate assessment mechanisms limit the ability to identify students who need early intervention. This contributes to higher dropout rates, especially in foundational programming courses (Venkatesan et al., 2024). Poorly designed assessments may unintentionally disadvantage students from underrepresented or underserved backgrounds, reinforcing existing disparities in CS education (Valiant, 2020). If assessments do not reflect true learning, educators may misinterpret student progress, potentially reinforcing ineffective teaching methods or misallocating resources. There are no widely accepted metrics to measure student performance in computer science, unlike mathematics or language arts. Students' achievements cannot be compared across schools, districts, or regions. Without standardized metrics, it is hard to ensure fairness or to gauge how well students are performing nationally or globally. This impacts

benchmarking, funding, and academic mobility. The lack of consistent assessment frameworks makes it challenging to evaluate the effectiveness of pedagogical interventions, hindering innovation in curriculum design and instructional strategies (Decker & McGill, 2019; Elliott Tew & Dorn, 2013; Parker et al., 2016).

New technologies, programming languages, and tools are constantly emerging in computer science. AI tools that offer tailored feedback or instruction depend on accurate assessment data. Without reliable performance metrics, these systems cannot function effectively. Assessments involving AI, code tracking, or behavioral analytics raise ethical concerns around surveillance, bias, and student data privacy, complicating their use in real educational settings. As a result, it can be challenging for educators and assessment developers to keep curriculum and assessments up-to-date (Kallia, 2017). Technology exposure and prior experience varies widely among students entering computer science programs. It can be challenging to design assessments that accommodate diversity while maintaining rigor and fairness (Repenning et al., 2016). Computer science evolves quickly, new programming languages, tools, and frameworks emerge often. Curricula and assessments can become outdated fast, making it hard for educators to stay current. Students with prior exposure may find assessments easier, while others struggle, not necessarily due to lack of ability, but lack of experience. This can lead to inaccurate evaluations of true potential. Students who feel unprepared or overwhelmed may lose confidence and drop out (Starks et al., 2024).

Time, expertise, and technology infrastructure are required for developing and implementing effective computer science assessments. Schools and educational institutions may lack these resources, limiting their ability to assess students accurately. Implementing sophisticated, practical, or AI-based assessments requires significant infrastructure, skilled personnel, and ongoing maintenance, such resources are not always available in educational institutions.

Computer science assessments often involve technology and data, raising **privacy, security, and ethical concerns**. It is challenging to design assessments that mitigate these concerns while providing meaningful insights into students' performance (Vakil, 2018). Assessments in computer science are susceptible to bias and stereotypes, especially for underrepresented groups. Assessments may be affected by stereotypes about marginalized students' abilities in STEM fields (O'Brien et al., 2025). Assessing students' readiness for careers or further education in computer science requires accurate assessments that reflect real-world tasks and challenges. The challenge is creating authentic assessments that balance authenticity with feasibility and reliability.

5.2.2. Challenges in student engagement

Complexity and abstract nature of concepts could be considered as one of the first issues in this category where computer science concepts can be complex and abstract, which may pose challenges for engaging students, especially those new to the field (Sinclair et al., 2015). CS concepts often require students to process multiple layers of abstraction simultaneously (e.g., syntax, logic, data structures). This demands a strong working memory and logical reasoning, which many beginners have not yet developed. Moreover, in the first years of CS education, unlike disciplines like biology or chemistry, CS lacks intuitive real-world analogs. Concepts like pointers, stack overflows, or runtime complexity are not easily visualized without scaffolding. Therefore, it is crucial to design engaging learning experiences that motivate students to invest time and energy in their studies in order to understand and internalize these concepts.

The computer science education student population has **diverse backgrounds** (Ihorn et al., 2020). Some students begin CS degrees with years of coding experience (via high school courses, online learning, or self-study), while others may have never written a line of code. This creates disparities in foundational knowledge and confidence levels. Students come from various school systems (e.g., AP Computer Science,

international curricula, no CS exposure at all), leading to inconsistent preparation and conceptual understanding. Access to computers, internet connectivity, and extracurricular learning resources varies widely, impacting students' readiness and engagement with course materials (Bui et al., 2023; Kirkpatrick & Mayfield, 2017). Furthermore, students may thrive in hands-on, project-based learning environments, while others may prefer structured lectures or independent study. Learning activities and assessments that cater to diverse learning strategies and preferences can be challenging, but essential for student engagement.

Technology-rich learning environments (TRLs) are often used in computer science education, such as coding platforms, virtual labs, and online discussion forums (Randolph et al., 2008). The integration of technology into the curriculum requires careful planning, technical expertise, and ongoing support. Various issues, compatibility issues, and accessibility concerns may hinder student engagement if not addressed proactively. Many TRLs require students to manage complex user interfaces, technical tools, and programming languages simultaneously. This cognitive overhead can overwhelm especially novice learners. TRLs often consist of multiple disconnected tools (e.g., IDEs, LMSs, simulators). Ensuring they work together without technical hiccups demands substantial effort from instructors and support staff. Many platforms provide static or delayed feedback, which hinders timely correction of misunderstandings, especially critical in subjects like programming. Finally, Using advanced systems like Git, command-line interfaces, or debugging tools can discourage learners who have not yet mastered foundational skills.

Traditional assessment methods, such as exams and quizzes, may not fully capture the depth of student engagement in computer science education, particularly for skills-based learning objectives. Traditional assessments like multiple-choice tests or written exams often measure recall and theoretical understanding. However, computer science is inherently practical and problem-solving oriented (e.g., coding, debugging, software design), which these methods fail to evaluate effectively. Traditional tools provide a snapshot of student outcomes but not the process (e.g., how a student approached a problem, whether they learned from errors, etc.). This misses key indicators of engagement and growth. Computer science students often vary in their learning speeds and strategies. Timed assessments or standardized formats do not account for this diversity, which can discourage students and distort performance data (Dikli, 2003; Yuan & Cao, 2019). In order to promote meaningful engagement and assess students' mastery of key concepts and skills, authentic, performance-based assessments are essential.

Many computer science courses cover a wide range of topics within a limited timeframe, leaving little room for in-depth exploration. CS curricula must cover many foundational areas such as, programming, algorithms, data structures, systems, databases, and theory. Each of which is critical to further learning. This breadth compresses available time per topic. New tools, languages, and frameworks constantly emerge, requiring updates to syllabi and training, often without increasing the time allocated to the course. Courses often prioritize summative exams and grading schemes focused on breadth, not depth, which discourages deeper engagement or student-led inquiry. Covering core content while allowing students to explore and be creative can be challenging. Engaging students requires effective **time management strategies and curriculum design** principles that prioritize essential concepts while allowing flexibility for student-driven learning.

Due to perceived difficulty, lack of confidence, and competing demands on students' time, computer science courses may experience **high attrition rates** (Decker & Simkins, 2016). CS involves abstract thinking, logic, and math-intensive content. For novices, especially those with weak prior exposure, this can be overwhelming. Early programming courses, such as CS1, often demand steep learning curves with little room for error. Students often experience "imposter syndrome" or feel incapable of mastering coding and algorithms. This is especially true among underrepresented groups who may not see role

models in the field. Instructors often cannot provide timely and personalized feedback to large cohorts, leaving students without support when they struggle. Educators must maintain student motivation and interest over time, especially during challenging periods. Mentoring, peer support networks, and personalized feedback can help students feel motivated and belong.

Promoting engagement and diversity in computer science requires equitable access to computer science education for all students. Systemic changes and proactive outreach are needed to address barriers to access, such as limited technology access. Promoting engagement and retention in computer science education requires inclusive learning environments in which students feel valued, supported, and empowered.

5.2.3. Challenges in identifying at-risk students

Despite the potential benefits of automatic content extraction for enhancing computer science education, several challenges remain. It remains difficult to teach computers to **understand the semantics of educational content**, especially in complex subjects like computer science (Atapattu Mudiyanse, 2015). Natural language processing (NLP) is required to extract meaningful information from programming assignments, code snippets, and technical descriptions.

From algorithms and data structures to software engineering and artificial intelligence, computer science covers a **wide range of topics**. Diverse courses and topics require robust automatic extraction systems. Computer science is a broad and interdisciplinary field encompassing areas such as algorithms, programming languages, databases, operating systems, cybersecurity, and artificial intelligence (The Joint Task Force on Computing Curricula, 2023). Each domain uses different terminology, problem-solving approaches, and pedagogical methods. Automatic content extraction systems must be sophisticated enough to adapt to this diversity. A one-size-fits-all approach is insufficient. Systems may perform well in extracting content from one topic (e.g., web development) but fail in another (e.g., formal methods). This leads to inconsistent content quality, making it difficult for educators to rely on such tools across the curriculum. It also limits scalability and restricts the application of automatic extraction to niche areas only.

The **accuracy and precision** of automatic content extraction are crucial, especially when dealing with technical concepts. Errors in content extraction could lead to significant misunderstandings or inaccuracies. Unlike general text, computer science materials often have precise definitions and syntax rules. A misplaced character in code or a slight variation in terminology (e.g., "stack" as a data structure vs. "stack" in memory management) can drastically change meaning. Therefore, automatic extraction systems must be extremely accurate and context-aware. Low accuracy can result in content that is not only useless but actively harmful. For instance, incorrect code snippets or misleading explanations can hinder comprehension or encourage incorrect practices. This undermines students' confidence, slows down their learning, and increases the cognitive load on instructors who must correct these mistakes.

To be useful for learners, automatically extracted content must be current and relevant. It is challenging to **keep up with the rapidly evolving field** of computer science and ensure that extracted content remains current. Technologies, languages, and tools in computer science evolve quickly. What is considered best practice today may be outdated within a few months. Automatic extraction systems must keep up with updates in documentation, APIs, and academic standards to ensure content remains relevant. Without mechanisms to regularly update extracted content, students may be exposed to deprecated tools, outdated syntaxes, or ineffective practices. This creates a gap between what students learn and what is expected in the industry, diminishing the practical value of the education they receive.

Students have **different learning preferences**. Individual learning strategies, preferences, and proficiency levels must be accommodated

by automatic content extraction systems. Students differ in how they learn best, for example, some prefer step-by-step explanations, others visualizations, and still others hands-on experimentation. Additionally, students enter courses with varying levels of prior knowledge and technical ability (Bui et al., 2023; Kirkpatrick & Mayfield, 2017). Automatic systems must cater to these differences to be effective. A lack of personalization leads to frustration or disengagement, particularly for students at either end of the proficiency spectrum. Advanced learners may find content too simplistic, while novices may be overwhelmed. This affects student satisfaction, retention rates, and ultimately, learning outcomes (Morrison et al., 2016).

Automated content extraction tools require seamless **integration with existing learning management systems** (LMS) or online learning platforms (Sunil Prakash et al., 2010). Compatibility and interoperability with existing educational technologies can be challenging (Alier et al., 2010).

Institutions and educators often rely on established LMS platforms such as Canvas, Moodle, or Blackboard. These systems have their own ecosystems and standards. Automatic content extraction tools must integrate smoothly with these platforms to be usable at scale. Without seamless integration, content extraction tools may create workflow disruptions, require redundant effort, or force educators to use multiple disjointed systems. This limits adoption and reduces the perceived value of such tools in institutional settings. Poor interoperability can also hinder data flow, affecting analytics and personalization features (Koucham et al., 2022).

Finally, ethical concerns such as data privacy, intellectual property rights, and algorithmic bias must be addressed in automatic content extraction. Protecting students' sensitive data and ensuring fair and unbiased content extraction processes are essential considerations. Automated systems must access and process educational data such as code submissions, text explanations, assessments. These data often contain sensitive personal or proprietary information. There are risks related to student privacy, academic integrity, and ownership of educational material. Additionally, biased algorithms may favor certain demographics or learning strategies. Failure to address these ethical issues could lead to privacy breaches, legal complications, or discrimination. For example, content extraction algorithms trained on non-diverse datasets might unintentionally reinforce stereotypes or marginalize certain students. Institutions may face liability issues, and students could lose trust in the platform (O'Brien et al., 2025).

5.2.4. Challenges in implementing interventions and recommendations

There are several challenges in identifying at-risk students in computer science education, such as considering a **range of indicators** beyond academic performance alone (Falkner & Falkner, 2012). Attendance, engagement levels, coursework participation, and even socioeconomic background can influence student success. Complexity can arise from integrating these diverse indicators.

Accurate and reliable data are essential for risk detection. Compiling and maintaining comprehensive data on student behavior, performance, and background can be challenging, particularly in large educational institutions with diverse systems and data sources. Ensuring **Data quality, consistency, and accessibility** are essential for accurate risk detection. In CS departments, student data is typically spread across multiple platforms, for example, learning management systems (LMS), code submission portals, attendance records, and peer collaboration tools. These systems may not be fully integrated, leading to incomplete, inconsistent, or even erroneous data. For instance, submission logs may be missing, or attendance may not be accurately recorded, especially in hybrid or online settings. This fragmentation makes it difficult to construct a unified view of student progress. Without high-quality, clean, and consistent data, any risk identification algorithm or system is prone to generating false positives or failing to detect students truly at risk (Hlosta et al., 2020; Jayaprakash et al., 2014).

Fairness and bias must be ensured in risk detection algorithms (Bellamy et al., 2019). Algorithms may inadvertently perpetuate existing inequalities or stereotypes, leading to unfair treatment of certain student groups. To address algorithmic bias, the risk detection system must be thoroughly tested, validated, and monitored. Despite extensive data, interpreting student risk signals can be challenging. Students' behavior and performance patterns must be analyzed nuancedly to differentiate transient academic difficulties from persistent issues. It is challenging to develop algorithms and models that accurately identify genuine risk factors (Bellamy et al., 2019). Predictive models built on historical data can unintentionally perpetuate existing inequalities within CS education. For example, if past data reflects the underperformance of underrepresented minorities, the algorithm may unfairly flag similar students in the future, reinforcing those disparities. Bias can also appear along lines of gender, language background, or learning style. An algorithm trained predominantly on data from one demographic group may not perform equitably across all students. As such, it is essential to perform regular audits, use inclusive datasets, and incorporate fairness-aware modeling techniques to ensure that no student group is unfairly disadvantaged (Baker & Hawn, 2022; Bird et al., 2025; Švábenský et al., 2024).

Detection systems often collect and analyze sensitive student data, raising **privacy and ethical concerns** (Rowe & Prior, 2022). Regulations such as General Data Protection Regulation (GDPR) and the Family Educational Rights and Privacy Act (FERPA) must be followed to ensure student privacy. In addition, informed consent and transparency should be considered when using predictive analytics in education. Monitoring students' digital footprints such as login times, code submissions, or participation patterns can feel invasive, especially if students are unaware of the extent of data collection. As aforementioned, regulations such as the GDPR and the FERPA impose legal requirements for transparency, consent, and data protection. Institutions must be cautious to ensure compliance, clearly communicate how data is used, and respect student autonomy. Ethical lapses in data usage can erode trust and deter students from fully engaging with digital platforms.

It takes time, personnel, and educational materials to identify at-risk students and provide appropriate support and interventions. Educational institutions may lack the resources to provide adequate support to all at-risk students, necessitating prioritization and strategic **allocation of resources**. Even when at-risk students are identified correctly, providing timely and effective support requires substantial resources. These may include academic advisors, mental health counselors, peer tutors, and targeted learning materials. However, many institutions, especially those with large enrollments or limited budgets, may lack the capacity to provide individualized support for every flagged student. This resource scarcity necessitates prioritization, which in itself raises equity and ethical questions. Institutions must carefully balance who receives support and how interventions are scaled without overlooking less visible forms of distress or disadvantage.

Different cohorts of students or educational contexts may have different **risk factors** for student success. Models and algorithms for detecting and adapting to these dynamic risk factors require continuous monitoring, evaluation, and refinement. Risk factors evolve depending on the student cohort, educational context, or even broader societal conditions. A first-year student may struggle with basic programming syntax, while a senior student might face stress related to capstone projects, job applications, or imposter syndrome. Predictive models and support systems must be regularly reviewed, updated, and validated to remain relevant and effective. A static approach risks becoming outdated and misaligned with actual student experiences.

5.2.5. Challenges with less use of generative AI and explainable AI

Leaving out Generative AI in computer science education can limit engagement and personalization. Without GenAI-powered tools, students rely on generic, static materials and miss out on dynamic exercises, real-time coding assistance, or tailored explanations. The gap particularly affects novice learners or those from diverse backgrounds, reducing motivation and potentially increasing dropout rates.

(Asdelen & Bodemer, 2025). Over-reliance on manual methods without GenAI may slow feedback, limiting students' ability to learn iteratively and reinforcing knowledge gaps (Vieriu & Petrea, 2025; Wu et al., 2025).

Furthermore, Explainable AI caters to trust and transparency issues. Students and educators may distrust or reject AI systems that seem like "black boxes", hindering their adoption and undermining their benefits (Bellas et al., 2025). Lack of explainability also makes it difficult to detect and mitigate algorithmic biases, risking unfair outcomes, particularly for underrepresented or marginalized CS students (Al-Zahrani, 2024; Chinta et al., 2024).

Deploying AI tools without transparency also increases ethical and legal risks. As a result of opaque design, institutions may unknowingly violate regulations like GDPR. Explainable AI is a foundational component of Responsible AI which is essential for ensuring privacy, fairness, and trust in educational contexts (Baker & Xiang, 2023).

Further, relying solely on conventional methods may contribute to a curriculum misaligned with current industry demands. Without GenAI's trend-analysis capabilities, CS programs may lag behind evolving technologies, limiting student readiness for the workforce and diminishing educational value (Capraro et al., 2024; Fan et al., 2025).

Additionally, overdependence on non-generative tools risks eroding critical thinking and creativity. Studies indicate that students who rely heavily on AI-generated solutions may show declines in cognitive engagement and reduced innovation over time (Capraro et al., 2024; Fan et al., 2025).

5.3. Opportunities

5.3.1. Opportunities in computer science education

One of the main opportunities in computer science education can be mentioned as **refining Curriculum and Instruction** where analyzing students' performance in computer science education highlights areas of strength and weakness in the curriculum. By using this information, educators can refine course content, adjust pacing, and identify topics that require additional attention. Effective teaching strategies can be developed by understanding student performance factors (Aven-dano et al., 2016). To enhance student understanding and retention, educators can incorporate techniques such as active learning, peer instruction, and formative assessment into their instructional methods. Another significant opportunity is **early intervention for At-Risk students** where by monitoring students' performance, educators can identify students who may be struggling academically. At-risk students can benefit from early intervention strategies (Al-Shabandar et al., 2019), such as tutoring, mentoring, or supplemental instruction. The next option is **resource allocation and support** in which staffing, funding, and technology investments can be informed by performance data analysis. These resources and services can assist schools and institutions in supporting student learning in computer science. By examining performance data by demographic factors such as gender, race, ethnicity, and socioeconomic status, we can identify disparities in achievement and promote diversity. Educators can foster academic excellence by addressing barriers to success and providing equitable opportunities for all students. Moreover, **accountability and quality of instruction** could be taken into consideration in which educators and institutions can be held accountable for student outcomes using performance data. Assessments, course evaluations, and other metrics can provide valuable insight into the quality of instruction. **Research and innovation** can be recognized as one of the main opportunities in computer science education in which new knowledge and innovative practices can be developed in computer science education through research. Researchers can identify effective interventions, pedagogical approaches, and technological tools by studying factors that influence learning and achievement. AI-powered code analysis tools can assess students' programming submissions beyond correctness by evaluating code structure, readability, efficiency, and logic. These systems, often

built on machine learning or rule-based engines, provide immediate and formative feedback, which is crucial for novice programmers. Such platforms (e.g., CodeBench, DeepCode, or AutoGrader with ML) reduce instructor workload while maintaining rigorous standards. They help identify misconceptions early, thereby supporting timely remediation and reducing dropout rates in foundational courses. Adaptive learning systems leverage AI algorithms to dynamically tailor the difficulty and nature of assessment tasks based on a learner's performance. By applying Bayesian Knowledge Tracing or reinforcement learning models, these systems can personalize learning trajectories and assessments in real-time. This approach accommodates students with varying levels of prior experience, offering challenge to advanced learners while supporting those who may struggle. Importantly, it maintains assessment rigor while promoting inclusivity. Machine learning models can analyze patterns in behavioral, performance, and demographic data to predict student risk levels with high accuracy. Integrating such predictive models into learning management systems (LMS) allows institutions to proactively identify students at risk of academic failure or disengagement. Early intervention strategies can then be deployed, such as personalized tutoring or mentoring. This data-driven approach enhances institutional capacity for student support and retention.

5.3.2. Opportunities regarding enhancing student engagement

Exploring students' engagement in computer science education opens up several possibilities. Understanding what engages students can lead to improved teaching methods and curriculum designs (Parsons & Taylor, 2011). By identifying what motivates students, educators can make the material more interesting and accessible. In computer science programs, particularly for underrepresented groups, recognizing factors that contribute to engagement can help to **enhance retention rates**. Educators can attract and retain a diverse range of students by addressing barriers to engagement. **Innovative instructional techniques** can be inspired by student engagement research (Alvarez-Bell et al., 2017). This can lead to the development of new instructional techniques, such as project-based learning, collaborative activities, and incorporating real-world applications. Educators can identify the best computer science resources and tools by studying what engages students. Educational software, online platforms, or interactive learning materials that are particularly engaging and effective could be included. Students' interests, backgrounds, and learning strategies can facilitate **personalized learning**. Teachers can provide students with opportunities to explore topics that align with their interests and abilities, thereby fostering greater engagement. Computer science education **prepares students for careers in technology**. Students can develop the skills, confidence, and passion they need to pursue higher education and careers in computer science and related fields. Research on student engagement in computer science education can **inform policy decisions** at the institutional, regional, and national levels. It can provide evidence-based recommendations for improving computer science education initiatives, allocating resources effectively, and promoting equity and diversity in STEM fields.

AI-driven platforms (like intelligent tutoring systems) can adjust difficulty levels, pace, and content delivery based on real-time student performance—keeping cognitive load manageable and engagement high. AI can also generate human-readable explanations for code (this is one of the features that CoPilot has under development), abstract concepts, or error messages using NLP, helping demystify abstract topics in plain language. Finally, AI can assess engagement levels through clickstream, emotion analysis, or behavioral tracking (using multimodal learning analysis (Sharma & Giannakos, 2020)). Based on this, it can introduce gamified elements or interventions to maintain attention. For reducing the differences based on the student diversity, AI-driven recommendation systems (e.g., Codebench, intelligent tutors) can suggest coding problems or resources that match each student's ability and learning goals. AI can help deliver content in various forms such as, videos, visualizations, simulations, text, interactive games.

These can cater to different learning strategies and increasing accessibility. AI tools can help flag inequities in assessment or participation, ensuring that learning environments are inclusive and no group is unintentionally disadvantaged. AI can personalize the learning experience by analyzing student behavior and performance. It can adjust difficulty levels, recommend resources, or suggest review topics based on individual progress to reduce the impact of new TRLs. AI can power chatbots or virtual assistants that help students troubleshoot errors, understand concepts, or navigate platforms using natural language queries. AI can identify students at risk of disengagement or failure by detecting early warning signs (e.g., inactivity, poor submission history) and alert instructors or trigger interventions. AI can forecast student performance trajectories, enabling proactive time allocation for high-impact topics or at-risk learners. This can support informed decisions about where to slow down or go deeper. AI-powered systems can automatically evaluate code for correctness, efficiency, and style. These systems can also provide formative feedback in real-time, helping students learn iteratively (e.g., CodeBench, JavaLUATOR). AI can track how students interact with learning platforms (e.g., time on task, debugging behavior, frequency of submissions) and use this to infer engagement, persistence, and learning strategies—insights that go far beyond right-or-wrong answers. AI can power virtual labs, gamified environments, or intelligent tutoring systems that simulate real-world programming tasks, assessing both the outcome and the process, which is a closer match to professional practice.

5.3.3. Opportunities with automated content extraction

A variety of ways can be made possible by automated content extraction in computer science education. **Personalized learning paths** can be created for individual students based on their learning goals, interests, and current level of understanding by automatically extracting relevant content from various sources such as textbooks, research papers, and online resources (Barbosa et al., 2023).

Augmented learning materials can be created using content extraction such as summaries, flashcards, quizzes, and interactive exercises from large volumes of text. This can help students grasp complex concepts more effectively and retain information better. Automatic content extraction can power **adaptive learning platforms** that dynamically adjust the difficulty and pace of learning materials based on students' performance and feedback. This ensures that each student receives tailored instruction and support. Extracting content from various sources and converting it into different formats such as audio, visual, or interactive formats can **improve accessibility** for students with disabilities, catering to diverse learning needs. Analyzing the extracted content and students' interactions with it can provide valuable insights into learning patterns, misconceptions, and areas where students may need additional support. This data can **inform instructional design and teaching strategies**. By automatically extracting relevant examples, case studies, and real-world applications of concepts, **learning experiences can be enriched**, making the content more engaging and relevant to students. Content extraction can assist educators in identifying the most relevant and up-to-date content (Adil et al., 2024) for **curriculum design**, ensuring that course materials are comprehensive, accurate, and aligned with industry trends and standards. For students whose primary language is not the language of instruction, content extraction can help translate and adapt learning materials, making them more accessible and comprehensible. Content extraction can aid in **generating assessment questions** and grading student responses, saving educators time and providing immediate feedback to students. Automatic content extraction can enable the **continuous updating** and improvement of learning materials to reflect the latest advancements and discoveries in the field of computer science, keeping the content relevant and up to date.

Researchers should focus on using fine-tuned large language models on computer science-specific corpora (e.g., textbooks, documentation, codebases) to improve their understanding of technical semantics.

Moreover, they can also use domain-specific ontologies (e.g., ACM Computing Classification System) to map educational content to structured knowledge graphs, improving semantic comprehension. Such methods enhance understanding of meaning, context, and intent in both natural language and code, enabling better summarization, question generation, and feedback. Given the breadth of computer science as a discipline, content extraction systems must be capable of adapting to a wide array of topics and terminologies. AI techniques such as topic modeling (e.g., Latent Dirichlet Allocation) and deep classification methods can help identify and organize content across subfields like machine learning, operating systems, and software engineering. These classifiers can be integrated with curriculum-aware frameworks that align extracted content with learning outcomes and course objectives, ensuring pedagogical relevance. Keeping extracted content current is another pressing concern, especially in a field that evolves as rapidly as computer science. AI can be leveraged to monitor and extract updates from authoritative sources, including official documentation, open-source repositories (e.g., GitHub), and developer forums (e.g., Stack Overflow). These real-time update systems can automatically detect deprecated features or new programming conventions and revise educational content accordingly. Automated content versioning systems can flag outdated material and recommend updates to instructors and learners, helping to maintain the relevance of educational resources.

5.3.4. Opportunities regarding supporting at-risk students

Detecting at-risk students in computer science education presents several opportunities for intervention and support, ensuring their success in the field. **Identifying at-risk students early** allows educators to intervene promptly. **Early interventions** can address underlying issues such as academic difficulties, lack of engagement, or personal challenges before they escalate (Horton, 2015). Once at-risk students are identified, **personalized support plans** can be developed to address their specific needs. This may include additional tutoring, mentoring, counseling, or modified learning strategies tailored to the student's learning style. By providing targeted support to at-risk students, **retention rates** in computer science programs can be improved. Retaining students who may otherwise drop out due to challenges can contribute to a more diverse and skilled workforce in the field. Detecting at-risk students can help **address disparities** in computer science education, particularly among underrepresented groups. By understanding the unique challenges faced by these students, educators can implement strategies to create a more inclusive learning environment. By utilizing data analytics and machine learning algorithms, educators can analyze various factors contributing to student success or failure in computer science education. This **data-driven** approach can provide valuable insights into patterns and trends, guiding future interventions and program improvements. Identifying at-risk students allows educators to focus on building not just academic skills but also crucial **soft skills** such as problem-solving, communication, and teamwork (Martin, 2019). Targeted interventions can help students develop a well-rounded skill set essential for success in the field. Understanding the challenges faced by at-risk students can inform **career counseling and guidance** efforts. Educators can provide resources and mentorship opportunities to help students explore career paths within computer science and navigate the job market effectively. **Creating a supportive community** within computer science programs can greatly benefit at-risk students. Peer support groups, networking events, and collaborative projects can foster a sense of belonging and motivation among students facing difficulties.

5.3.5. Providing recommendations and interventions

Providing recommendations and interventions in computer science education can enhance learning outcomes and prepare students for a rapidly changing field. First, making sure **computer science curricula are aligned with industry trends** and best practices (Kang et al., 2018). In order to equip students with relevant skills, key

concepts, programming languages, and technologies should be identified. Second, making computer science concepts more accessible and **engaging for students of all levels**. It could involve project-based learning, interactive simulations, or gamification. Such interventions could also promoting diversity and inclusion in computer science education by addressing barriers. Implement **outreach programs, provide mentorship opportunities**, and foster a supportive learning environment. Recommendations and interventions can also help in assessing and evaluating students' understanding of computer science concepts (Buffum et al., 2015). To measure overall learning outcomes, both formative and summative assessments will be developed. Moreover, **custom software applications**, interactive tutorials, or online learning modules tailored to specific topics or objectives could be developed. Developing recommendations for teacher training programs and professional development initiatives could ensure educators have the necessary skills and knowledge to effectively teach computer science. Pedagogy and content knowledge workshops, seminars, or online courses might also be included here. Ensuring that **educators have the necessary skills** and knowledge to teach computer science effectively is crucial for the success of students.

To effectively address the complex task of identifying and supporting at-risk students in computer science education, a variety of AI-based interventions can be implemented. One foundational approach involves the use of multimodal machine learning models that integrate diverse data sources such as grades, attendance, learning management system (LMS) activity, forum participation, and code submission logs. These models provide a more holistic understanding of student performance and engagement by identifying risk signals that go beyond academic scores alone. Techniques such as ensemble models, neural networks, and time-series analysis allow for the modeling of complex behavioral patterns, which are particularly relevant in CS education where self-paced learning, irregular submission habits, and independent study are common.

To personalize intervention even further, anomaly detection algorithms can be used to identify deviations in a student's typical behavior. Instead of comparing a student to their peers, these models such as, isolation forests, autoencoders, or one-class SVMs, detect individual changes, like sudden drops in activity or performance, that may signal emerging risk. This approach accounts for diverse learning strategies and schedules that are common in CS courses, especially in online or asynchronous environments, offering a more personalized view of student risk.

Transparency and interpretability are equally important for gaining stakeholder trust in AI systems (Chaushi et al., 2023; Khosravi et al., 2022; Marconi et al., 2020; Rachha & Seyam, 2023; Raees et al., 2024; Zoppis et al., 2020). Explainable AI (XAI) tools such as SHAP (Shapley Additive Explanations) or LIME (Local Interpretable Model-agnostic Explanations) provide insights into how a risk prediction is made, showing which features influenced the decision and to what extent. This allows instructors, advisors, and even students themselves to understand why they were flagged, increasing the transparency and accountability of automated systems.

Given the sensitivity of student data, AI systems must be developed with privacy-preserving techniques (Chaushi et al., 2023; Khosravi et al., 2022; Marconi et al., 2020; Rachha & Seyam, 2023; Raees et al., 2024; Zoppis et al., 2020). Methods like federated learning enable models to be trained on decentralized data across departments or institutions without sharing raw student data, while differential privacy introduces statistical noise to protect individual identities. These approaches help institutions comply with regulations such as GDPR and FERPA and ensure that ethical standards are upheld in the deployment of predictive analytics in education.

5.3.6. Evaluating effectiveness

Evaluating the effectiveness of different interventions and educational approaches in computer science education can be done through new research studies. The process involves collecting data, analyzing results, and disseminating findings to inform future policies and practices.

Longitudinal studies are crucial for understanding the long-term effectiveness of AI-supported retention strategies in education. Unlike short-term assessments, these studies enable researchers to track how AI interventions — such as personalized feedback, early-warning systems, and adaptive learning platforms — impact student retention across multiple semesters or academic years. They reveal whether initial improvements in engagement or performance lead to sustained retention and graduation rates.

Moreover, longitudinal research helps uncover how students' learning behaviors, problem-solving strategies, and perceptions of AI evolve over time. It allows educators to assess cumulative effects of AI tools, such as fostering self-regulated learning or increasing trust in AI-generated feedback. Longitudinal studies are also vital for detecting model drift, ensuring AI systems remain accurate and effective as student behaviors, curricula, and learning environments change. This supports the continuous improvement and adaptability of AI systems.

From an equity perspective, long-term studies are essential to evaluate whether AI interventions effectively reduce retention disparities among underrepresented or disadvantaged student groups. They also help monitor potential biases that may emerge after prolonged AI deployment. Furthermore, longitudinal research captures broader psychological and academic outcomes, including student confidence, motivation, academic resilience, and well-being.

Design-Based Research (DBR) plays a critical role in developing effective human-AI co-teaching models for computer science (CS) education. CS classrooms are complex learning environments where students, instructors, and AI systems interact dynamically. DBR is uniquely suited to navigate this complexity by enabling iterative design, testing, and refinement of AI-driven instructional models in real-world settings. Unlike controlled experimental approaches, DBR emphasizes context-sensitive solutions, allowing researchers to adapt AI co-teaching systems based on actual classroom needs.

One of the main advantages of DBR is its collaborative and cyclical nature. Teachers, researchers, and AI developers co-design solutions, ensuring that AI systems not only deliver accurate content support but also align with pedagogical goals, classroom workflows, and students' learning strategies. This iterative process is essential for fine-tuning critical aspects of human-AI collaboration, such as intervention timing, feedback clarity, and balancing instructional control between human and machine.

Additionally, DBR fosters the development of practice-validated design principles that guide scalable implementation of co-teaching models across diverse educational contexts. It also addresses socio-technical challenges like trust, transparency, fairness, and privacy by embedding ethical considerations into every design cycle. This is vital in CS education, where algorithmic bias or opaque AI systems could inadvertently disadvantage certain student groups.

Finally, the AI-based tools can also be tested and evaluated for the explanations (XAI modules). In certain recent work it has been shown that XAI reduces cognitive load (Wang, 2024), fosters trust in the systems (Hoq et al., 2023), increases the meaningful engagement with the system (Schmude et al., 2024). Empirical evidence suggests that explainable AI methods can meaningfully support novice programmers through reduced cognitive load, increased transparency, and higher trust. Visual, modular, and stakeholder-aware explanations show promise in helping learners engage more deeply, while performance prediction models gain credibility when their decisions are interpretable. Integrating these approaches can make AI tools in

Table A.1
The Quality metrics example.

C1. Is the research problem clearly addressed in the study? The selected articles have clearly reflected the main problem that the article is tackling while objectives and research questions are articulated very well. As an example, Kaur and Chahal (2023) identifies gaps in current research as highlighted by the statement “ <i>Despite the benefits to students’ academic success, LADs in introductory programming have received relatively little attention in the past technological revolution in education.</i> ”
C2. Are the research objectives clearly articulated in the study? When it comes to the research aims, the chosen academic papers clearly and effectively communicate the main objectives and goals of their scholarly contributions. The research goal introduced by Kaur and Chahal (2023) is articulated by the following statement: “The overall objective is to support CSE students with detailed insight into their non-cognitive (e.g., personality, learning motivation, and class participation) factors and adaptive feedback and recommendations for the factors’ contribution to their programming performance.”
C3. Is the context in which the research was conducted described in sufficient detail? A thorough description of the educational environment and learning context is expected in the publications. The study referenced in Kaur and Chahal (2023) precisely mentions that <i>the subjects included 280 undergraduate CSE students who had completed their initial programming course (C Language) in the first year of their undergraduate studies.</i>
C4. Was the research design suitable for addressing the aims of the study? The research design must be well-aligned with the requirements of the research questions. For instance, the study in Kaur and Chahal (2023) <i>concentrates specifically on enhancing the programming skills and motivation of first-year Computer Science and Engineering (CSE) students by creating a dashboard that employs predictive and prescriptive analytics.</i>
C5. Does the study clearly define the research methods, including subjects, instruments, data collection, and data analysis? The research methods ought to be explicitly detailed in the document. In Kaur and Chahal (2023) , the subjects (a description of the student population), data collection methods (surveys, students’ CPGA data from the previous semester), and data analysis techniques (qualitative analysis by the committee and laypersons) are meticulously outlined.
C6. Were rigorous data analysis techniques employed in the study? The data analysis is thoroughly presented to ensure that it addresses the research questions effectively. Kaur and Chahal (2023) utilizes machine learning techniques for predicting student performance, with the methods for explainability detailed comprehensively. The efficiency and utility of the dashboard are evaluated in two stages: through committee and laypersons evaluations. Furthermore, the qualitative analysis of the survey responses is elaborated upon and visualized using box plots.
C7. Are the research findings presented clearly? Findings should be communicated clearly and succinctly, with statements summarizing the principal results. Kaur and Chahal (2023) stated that (A) Committee members found the dashboard to be clear and user-friendly. (B) Both committee members and laypersons found the dashboard effective for monitoring factors influencing students’ learning during the academic period. (C) The dashboard effectively reflects the current and potential future learning scenarios of CSE students in programming. (D) There is a need for the dashboard to include optional, more detailed descriptions of each factor to provide students with a clearer understanding of each element.
C8. Does the study offer valuable contributions to research or practical applications? The contributions included in the Systematic Literature Review (SLR) must clearly demonstrate implications for research and/or practice. Section 5.2 of Kaur and Chahal (2023) is dedicated to the discussion on the implications of the study.

programming education more effective, equitable, and trustworthy.

6. Conclusion

This systematic literature review offers a comprehensive examination of the integration of artificial intelligence (AI) and machine learning (ML) in undergraduate computer science education (CSE). By analyzing 40 peer-reviewed papers, this review identifies the predominant use of “Log and trace data” followed by “CPGA and academic performance” and “Programming and code-related data” in developing AI-driven educational tools, with Supervised Learning, Ensemble Methods, and Deep Learning being the most extensively employed AI methodologies.

However, the review highlights a gap in the exploration of Generative AI and Explainable AI (XAI), presenting an opportunity for future research to delve into these underutilized areas.

The findings indicate a significant focus on programming education, which has gained increased attention post-pandemic, reflecting the global shift in educational needs and the potential of AI and ML to enhance educational outcomes.

Moreover, the review has categorized the main research objectives and findings of the papers into six main themes namely learning behavior, learning performance and outcomes, at risk learner, interventions, hints or recommendations, and automatic content extraction.

Furthermore, despite the promising applications of AI and ML in CSE, the review identifies several challenges, including the complexity of assessing student performance, ensuring student engagement, and addressing ethical concerns related to data privacy and algorithmic bias.

The review also highlights several opportunities, such as the potential for AI-driven personalized learning experiences, early identification & support for at-risk students, and the development of adaptive learning environments that cater to diverse learner needs.

CRediT authorship contribution statement

Anahita Golrang: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Kshitij Sharma:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

Generative AI tools (ChatGPT, OpenAI) were used solely for language editing purposes, including grammar checking, rephrasing, and improving clarity of expression. No AI tools were used for generating research ideas, data analysis, or substantive content. The authors take full responsibility for the content and integrity of this manuscript.

Appendix

(see [Table A.2](#)).

Table A.2
Comparison of Kitchenham & Charters (2007) and PRISMA (2020) guidelines for systematic reviews.

Aspect	Kitchenham & Charters (2007)	PRISMA (2020)
Domain Origin Purpose	Software Engineering Guide for conducting SLRs in software engineering (adapted from medical guidelines)	Health Sciences Standard for reporting systematic reviews and meta-analyses
Focus	Entire SLR process (planning, conducting, reporting)	Mainly on reporting systematic reviews transparently
Structure	3 Main Phases: Planning, Conducting, Reporting	Structured checklist + flow diagram focused on final reporting
Planning Emphasis	Strong emphasis on protocol development, research questions, and review team roles	Planning assumed but not detailed; focus is on protocol registration and transparency
Research Questions	Based on PICOC (Population, Intervention, Comparison, Outcome, Context)	Based on PICO (Population, Intervention, Comparison, Outcome)
Search Strategy	Highly detailed, tailored to software engineering databases (IEEE Xplore, ACM, etc.)	Equally detailed; tailored more to medical/health science databases (PubMed, Embase)
Selection & Screening	Involves inclusion/exclusion criteria, dual screening, quality assessment, and data extraction	Similar processes, but more emphasis on risk of bias and reporting transparency
Quality Assessment	Strong focus using domain-specific quality checklists	Systematic tools like ROBIS , GRADE , or Cochrane Risk of Bias
Synthesis	Often qualitative or thematic; quantitative meta-analysis less common in SE	Often includes meta-analysis with statistical synthesis
Output Format	Research-style report with clear method transparency	Standardized publication-ready report format with abstract, methods, results, discussion
Flow Diagram	Optional; less formalized, can be included	Required; standardized PRISMA flow diagram for study selection

References

Abildinova, G., Sembayev, T., & Mukhtarkyzy, K. (2024). Enhancing computer science education: student insights on active learning and digital integration. *World Transactions on Engineering and Technology Education*, 22(4), 299–305.

Adil, H. M., Ali, S., Sultan, M., Ashiq, M., & Rafiq, M. (2024). Open education resources' benefits and challenges in the academic world: a systematic review. *Global Knowledge, Memory and Communication*, 73(3), 274–291.

Ahmad, K., Iqbal, W., El-Hassan, A., Qadir, J., Benhaddou, D., Ayyash, M., & Al-Fuqaha, A. (2023). Data-driven artificial intelligence in education: A comprehensive review. *IEEE Transactions on Learning Technologies*.

Al-Shabandar, R., Hussain, A. J., Liatsis, P., & Keight, R. (2019). Detecting at-risk students with early interventions using machine learning techniques. *IEEE Access*, 7, 149464–149478.

Al-Zahrani, A. M. (2024). Unveiling the shadows: Beyond the hype of AI in education. *Heliyon*, 10(9).

Alalawi, K., Athauda, R., & Chiong, R. (2023). Contextualizing the current state of research on the use of machine learning for student performance prediction: A systematic literature review. *Engineering Reports*, Article e12699.

Albreiki, B., Habuza, T., Palakkal, N., & Zaki, N. (2023). Clustering-based knowledge graphs and entity-relation representation improves the detection of at risk students. *Education and Information Technologies*, 1–30.

Albreiki, B., Zaki, N., & Alashwal, H. (2021). A systematic literature review of student performance prediction using machine learning techniques. *Education Sciences*, 11(9), 552.

Algarni, S., & Sheldon, F. (2023). Systematic review of recommendation systems for course selection. *Machine Learning and Knowledge Extraction*, 5(2), 560–596.

Alhitmi, H. K., Mardiah, A., Al-Sulaiti, K. I., & Abbas, J. (2024). Data security and privacy concerns of AI-driven marketing in the context of economics and business field: an exploration into possible solutions. *Cogent Business & Management*, 11(1), Article 2393743.

Alier, M. F., Guerrero, M. J. C., González, M. Á. C., Peñalvo, F. J. G., & Severance, C. (2010). Interoperability for LMS: the missing piece to become the common place for e-learning innovation. *International Journal of Knowledge and Learning*, 6(2–3), 130–141.

Alvarez-Bell, R. M., Wirtz, D., Bian, H., et al. (2017). Identifying keys to success in innovative teaching: Student engagement and instructional practices as predictors of student learning in a course using a team-based learning approach. *Teaching and Learning Inquiry*, 5(2), 128–146.

Apiola, M., Saqr, M., & López-Pernas, S. (2023). The evolving themes of computing education research: Trends, topic models, and emerging research. *Past, Present and Future of Computing Education Research: A Global Perspective*, 151–169.

Asdelen, O., & Bodemer, D. (2025). Generative AI in the classroom: Effects of context-personalized learning material and tasks on motivation and performance. *International Journal of Artificial Intelligence in Education*, 1–22.

Asthana, P., Mishra, S., Gupta, N., Derawi, M., & Kumar, A. (2023). Prediction of student's performance with learning coefficients using regression based machine learning models. *IEEE Access*.

Atalla, S., Daradkeh, M., Gawanmeh, A., Khalil, H., Mansoor, W., Miniaoui, S., & Himeur, Y. (2023). An intelligent recommendation system for automating academic advising based on curriculum analysis and performance modeling. *Mathematics*, 11(5), 1098.

Atapattu Mudiyansele, T. D. A. (2015). *A computational model for task-adapted knowledge organisation: Improving learning through concept maps extracted from lecture slides* (Ph.D. thesis).

Avendano, R., Barrera-Osorio, F., Parra, S. N., & Vever, F. (2016). Understanding student performance beyond traditional factors: Evidence from PISA.

Awasthi, R., & Shukla, A. (2025). A data-driven framework based on machine learning approaches for restructuring computer science courses. *International Journal of Environmental Sciences*, 11(8s), 736–742.

Baker, R. S., & Hawn, A. (2022). Algorithmic bias in education. *International Journal of Artificial Intelligence in Education*, 32(4), 1052–1092.

Baker, S., & Xiang, W. (2023). Explainable AI is responsible AI: How explainability creates trustworthy and socially responsible artificial intelligence. *arXiv preprint arXiv:2312.01555*.

Barbosa, P. L. S., Carmo, R. A. F. d., Gomes, J. P., & Viana, W. (2023). Adaptive learning in computer science education: A scoping review. *Education and Information Technologies*, 1–50.

Bassner, P., Frankford, E., & Krusche, S. (2024). Iris: An ai-driven virtual tutor for computer science education. In *Proceedings of the 2024 on innovation and technology in computer science education V. 1* (pp. 394–400).

Baykasoğlu, A., Özbel, B. K., Dudaklı, N., Subulan, K., & Şenol, M. E. (2018). Process mining based approach to performance evaluation in computer-aided examinations. *Computer Applications in Engineering Education*, 26(5), 1841–1861.

Bellamy, R. K., Dey, K., Hind, M., Hoffman, S. C., Houde, S., Kannan, K., Lohia, P., Martino, J., Mehta, S., Mojsilović, A., et al. (2019). AI fairness 360: An extensible toolkit for detecting and mitigating algorithmic bias. *IBM Journal of Research and Development*, 63(4/5), 1–4.

Bellas, F., Ooeg, J., Roddeck, L., Rashheed, H. A., Skenduli, M. P., Masdoum, F., Zainuddin, N. b., Gori, J. N., Costello, E., Kralj, L., et al. (2025). Explainable AI in education: fostering human oversight and shared responsibility.

Berland, M., Baker, R. S., & Blikstein, P. (2014). Educational data mining and learning analytics: Applications to constructionist research. *Technology, Knowledge and Learning*, 19, 205–220.

Bhosale, D., & Sawane, M. (2021). Student's performance prediction using hybrid optimization algorithm-based map reduce framework. *AIJR Proceedings*, 304–308.

Bird, K. A., Castleman, B. L., & Song, Y. (2025). Are algorithms biased in education? Exploring racial bias in predicting community college student success. *Journal of Policy Analysis and Management*, 44(2), 379–402.

- Blikstein, P., Worsley, M., Piech, C., Sahami, M., Cooper, S., & Koller, D. (2014). Programming pluralism: Using learning analytics to detect patterns in the learning of computer programming. *Journal of the Learning Sciences*, 23(4), 561–599.
- Bond, M., Khosravi, H., De Laat, M., Bergdahl, N., Negrea, V., Oxley, E., Pham, P., Chong, S. W., & Siemens, G. (2024). A meta systematic review of artificial intelligence in higher education: a call for increased ethics, collaboration, and rigour. *International Journal of Educational Technology in Higher Education*, 21(1), 4.
- Borys, K., Schmitt, Y. A., Nauta, M., Seifert, C., Krämer, N., Friedrich, C. M., & Nensa, F. (2023). Explainable AI in medical imaging: An overview for clinical practitioners—Beyond saliency-based XAI approaches. *European Journal of Radiology*, 162, Article 110786.
- Buffum, P. S., Lobene, E. V., Frankosky, M. H., Boyer, K. E., Wiebe, E. N., & Lester, J. C. (2015). A practical guide to developing and validating computer science knowledge assessments with application to middle school. In *Proceedings of the 46th ACM technical symposium on computer science education* (pp. 622–627).
- Bui, G., Sibia, N., Zavaleta Bernuy, A., Liut, M., & Petersen, A. (2023). Prior programming experience: A persistent performance gap in CS1 and CS2. In *Proceedings of the 54th ACM technical symposium on computer science education V. 1* (pp. 889–895).
- Buschetto, Macarini, L. A., Cechinel, C., Batista Machado, M. F., Faria Culmant Ramos, V., & Munoz, R. (2019). Predicting students success in blended learning—evaluating different interactions inside learning management systems. *Applied Sciences*, 9(24), 5523.
- Çağataylı, M., & Çelebi, E. (2022). Estimating academic success in higher education using big five personality traits, a machine learning approach. *Arabian Journal for Science and Engineering*, 47(2), 1289–1298.
- Calderon, I., Silva, W., & Feitosa, E. (2023). Active learning methodologies for teaching programming in undergraduate courses: A systematic mapping study. *Informatics in Education*.
- Capraro, V., Lentsch, A., Acemoglu, D., Akgun, S., Akhmedova, A., Bilancini, E., & Viale, R. (2024). The impact of generative artificial intelligence on socioeconomic inequalities and policy making. *PNAS Nexus*, 3(6), pgae191.
- Carmen, C. (2022). Data analysis of short-term and long-term online activities in LMS. *TEM Journal*, 11(2), 497–505.
- Carneiro, M. G., Dutra, B. L., Paiva, J. G. S., Gabriel, P. H. R., & Araújo, R. D. (2022). Educational data mining to support identification and prevention of academic retention and dropout: a case study in introductory programming. *Revista Brasileira de Informática na Educação*, 30, 379–395.
- Casper, A. A., Rambo-Hernandez, K. E., Park, S., & Atadero, R. A. (2022). The impact of emergency remote learning on students in engineering and computer science in the United States: An analysis of four universities. *Journal of Engineering Education*, 111(3), 703–728.
- Chandrasekara, S., Hewavitharana, D., Weerasinghe, M., Gayasri, B., Wijendra, D., & De Silva, D. (2025). Gamifying coding education for beginners: Empowering learners with HTML, CSS and JavaScript. In *2025 international research conference on smart computing and systems engineering* (pp. 1–7). IEEE.
- Charytanowicz, M., Zoła, M., & Suszyński, W. (2024). The impact of the COVID-19 pandemic on higher education: Assessment of student performance in computer science. *Plos One*, 19(8), Article e0305763.
- Chaush, B. A., Selimi, B., Chaush, A., & Apostolova, M. (2023). Explainable artificial intelligence in education: A comprehensive review. In *World conference on explainable artificial intelligence* (pp. 48–71). Springer.
- Chen, C.-H., & Chang, C.-L. (2024). Effectiveness of AI-assisted game-based learning on science learning outcomes, intrinsic motivation, cognitive load, and learning behavior. *Education and Information Technologies*, 29(14), 18621–18642.
- Chen, L., Chen, P., & Lin, Z. (2020). Artificial intelligence in education: A review. *Ieee Access*, 8, 75264–75278.
- Chen, X., Liang, J., Kuo, R., & Tang, J. (2025). Exploring COVID-19's impact on computer science education using student reviews. In *2025 intermountain engineering, chen2025exploringTechnology and computing* (pp. 1–6). IEEE.
- Chen, X., Zou, D., & Xie, H. (2022). A decade of learning analytics: Structural topic modeling based bibliometric analysis. *Education and Information Technologies*, 27(8), 10517–10561.
- Chen, X., Zou, D., Xie, H., Cheng, G., & Liu, C. (2022). Two decades of artificial intelligence in education. *Educational Technology & Society*, 25(1), 28–47.
- Chinta, S. V., Wang, Z., Yin, Z., Hoang, N., Gonzalez, M., Quy, T. L., & Zhang, W. (2024). FairAIED: Navigating fairness, bias, and ethics in educational AI applications. *arXiv preprint arXiv:2407.18745*.
- Chu, H.-C., Hwang, G.-H., Tu, Y.-F., & Yang, K.-H. (2022). Roles and research trends of artificial intelligence in higher education: A systematic review of the top 50 most-cited articles. *Australasian Journal of Educational Technology*, 38(3), 22–42.
- Chung, C.-Y., & Hsiao, I.-H. (2020). Investigating patterns of study persistence on self-assessment platform of programming problem-solving. In *Proceedings of the 51st ACM technical symposium on computer science education* (pp. 162–168).
- Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37–46.
- Colburn, T., & Shute, G. (2007). Abstraction in computer science. *Minds and Machines*, 17(2), 169–184.
- Conceição, W., Holanda, M., Macedo, F., Ishikawa, E., Nunes, V. T., & Da Silva, D. (2022). Automatic feedback in the teaching of programming in undergraduate courses: a literature mapping. In *2022 IEEE frontiers in education conference* (pp. 1–9). IEEE.
- Coroiu, A., Călin, A., Găceanu, R., & Vescan, A. (2024). Advancements, challenges, and emerging trends in computer science education: A systematic literature review of academic and professional learning. In *2024 IEEE international conference on software analysis, evolution and reengineering-companion* (pp. 1–12). IEEE.
- Crompton, H., & Burke, D. (2023). Artificial intelligence in higher education: the state of the field. *International Journal of Educational Technology in Higher Education*, 20(1), 1–22.
- de Melo-Minardi, R. C., de Melo, E. C., & Bastos, L. L. (2022). OnlineBioinfo: Leveraging the teaching of programming skills to life science students through learning analytics. In *Frontiers in education: vol. 7*, Frontiers, Article 727019.
- van de Sande, D., van Bommel, J., Fung Fen Chung, E., Gommers, D., & van Genderen, M. E. (2022). Algorithmic fairness audits in intensive care medicine: artificial intelligence for all? *Critical Care*, 26(1), 315.
- Debut (2020). Degree dropouts rates summed up. <https://debut.careers/degree-dropouts/>.
- Decker, A., & McGill, M. M. (2019). A topical review of evaluation instruments for computing education. In *Proceedings of the 50th ACM technical symposium on computer science education* (pp. 558–564).
- Decker, A., & Simkins, D. (2016). Uncovering difficulties in learning for the intermediate programmer. In *2016 IEEE frontiers in education conference* (pp. 1–8). IEEE.
- Dever, D. A., Azevedo, R., Cloude, E. B., & Wiedbusch, M. (2020). The impact of autonomy and types of informational text presentations in game-based environments on learning: converging multi-channel processes data and learning outcomes. *International Journal of Artificial Intelligence in Education*, 30, 581–615.
- Dikli, S. (2003). Assessment at a distance: Traditional vs. alternative assessments. *Turkish Online Journal of Educational Technology-TOJET*, 2(3), 13–19.
- Dybå, T., & Dingsøyr, T. (2008). Empirical studies of agile software development: A systematic review. *Information and Software Technology*, 50(9–10), 833–859.
- Elegbeleye, F., & Isong, B. (2025). A systematic review of challenges in teaching and learning computer programming modules. *The Indonesian Journal of Computer Science*, 14(1).
- Elliott Tew, A., & Dorn, B. (2013). The case for validated tools in computer science education research. 46(9).
- Elmoazen, R., Saqr, M., Khalil, M., & Wasson, B. (2023). Learning analytics in virtual laboratories: a systematic literature review of empirical research. *Smart Learning Environments*, 10(1), 23.
- Espirito, D. J., & Todorovic, R. (2020). Increasing diversity and student success in engineering and computer science through contextualized practices. In *2020 ASEE virtual annual conference content access*.
- Falkner, N. J., & Falkner, K. E. (2012). A fast measure for identifying at-risk students in computer science. In *Proceedings of the ninth annual international conference on international computing education research* (pp. 55–62).
- Fan, L., Deng, K., & Liu, F. (2025). Educational impacts of generative artificial intelligence on learning and performance of engineering students in China. *Scientific Reports*, 15(1), 26521.
- Fang, C., Miao, N., Srivastav, S., Liu, J., Zhang, R., Fang, R., Tsang, R., Nazari, N., Wang, H., Homayoun, H., et al. (2024). Large language models for code analysis: Do {LLMs} really do their job? In *33rd USENIX security symposium* (pp. 829–846).
- Fonseca, S. C., Pereira, F. D., Oliveira, E. H., Oliveira, D. B., Carvalho, L. S., & Cristea, A. I. (2020). Automatic subject-based contextualisation of programming assignment lists. *International Educational Data Mining Society*.
- Francisco, R. E., & de Oliveira Silva, F. (2022). Intelligent tutoring system for computer science education and the use of artificial intelligence: A literature review. *CSEdu (1)*, 338–345.
- Franklin, D., Denny, P., Gonzalez-Maldonado, D. A., & Tran, M. (2025). *Generative AI in computer science education: Challenges and opportunities*. Cambridge University Press.
- Gaitantzi, A., & Kazanidis, I. (2025). The role of artificial intelligence in computer science education: A systematic review with a focus on database instruction. *Applied Sciences*, 15(7), 3960.
- Gao, G., Marwan, S., & Price, T. W. (2021). Early performance prediction using interpretable patterns in programming process data. In *Proceedings of the 52nd ACM technical symposium on computer science education* (pp. 342–348).
- Goode, J., Flapan, J., & Margolis, J. (2018). Computer science for all. *Diversifying Digital Learning: Online Literacy and Educational Opportunity* (2018), 45, 2018.
- Goundar, S., Deb, A., Lal, G., & Naseem, M. (2022). Using online student interactions to predict performance in a first-year computing science course. *Technology, Pedagogy and Education*, 31(4), 451–469.
- Guo, T., Bai, X., Tian, X., Firmin, S., & Xia, F. (2022). Educational anomaly analytics: features, methods, and challenges. *Frontiers in Big Data*, 4, Article 811840.
- Haleem, A., Javaid, M., Qadri, M. A., & Suman, R. (2022). Understanding the role of digital technologies in education: A review. *Sustainable Operations and Computers*, 3, 275–285.
- Hawltischek, A., Köppen, V., Dietrich, A., & Zug, S. (2020). Drop-out in programming courses—prediction and prevention. *Journal of Applied Research in Higher Education*, 12(1), 124–136.

- Heikkinen, S., Saqr, M., Malmberg, J., & Tedre, M. (2023). Supporting self-regulated learning with learning analytics interventions—a systematic literature review. *Education and Information Technologies*, 28(3), 3059–3088.
- Hellas, A., Ihtantola, P., Petersen, A., Ajanovski, V. V., Gutica, M., Hynninen, T., Knutas, A., Leinonen, J., Messom, C., & Liao, S. N. (2018). Predicting academic performance: a systematic literature review. In *Proceedings companion of the 23rd annual ACM conference on innovation and technology in computer science education* (pp. 175–199).
- Herman, D. S., Reece, J. T., & Weissman, G. E. (2024). Lessons for local oversight of AI in medicine from the regulation of clinical laboratory testing. *Npj Digital Medicine*, 7(1), 359.
- Higher Education Statistics Agency (HESA) (2017). Computer science undergraduates most likely to drop out. Retrieved from <https://www.computerweekly.com/news/252467745/Computer-science-undergraduates-most-likely-to-drop-out>.
- Hlosta, M., Papathoma, T., & Herodotou, C. (2020). Explaining errors in predictions of at-risk students in distance learning education. In *International conference on artificial intelligence in education* (pp. 119–123). Springer.
- Hoel, T., Griffiths, D., & Chen, W. (2017). The influence of data protection and privacy frameworks on the design of learning analytics systems. In *Proceedings of the seventh international learning analytics & knowledge conference* (pp. 243–252).
- Hoo, M., Brusilovsky, P., & Akram, B. (2023). Analysis of an explainable student performance prediction model in an introductory programming course. In *The 16th international conference on educational data mining*.
- Horton, J. (2015). Identifying at-risk factors that affect college student success. *International Journal of Process Education*, 7(1), 83–101.
- Huang, L. (2023). Ethics of artificial intelligence in education: Student privacy and data protection. *Science Insights Education Frontiers*, 16(2), 2577–2587.
- Huang, L., Liang, M., Xiong, Y., Wu, X., & Lim, C. P. (2024). A systematic review of technology-enabled teacher professional development during COVID-19 pandemic. *Computers & Education*, 223, Article 105168.
- Ibe, N. A., Howsmon, R., Penney, L., Granor, N., DeLyser, L. A., & Wang, K. (2018). Reflections of a diversity, equity, and inclusion working group based on data from a national CS education program. In *Proceedings of the 49th ACM technical symposium on computer science education* (pp. 711–716).
- Ihorn, S., Yoon, I., & Kulkarni, A. (2020). Student psychological factors and diversity in computer science education. In *Proceedings of the 51st ACM technical symposium on computer science education* (pp. 1380–1380).
- Jayaprakash, S. M., Moody, E. W., Lauria, E. J., Regan, J. R., & Baron, J. D. (2014). Early alert of academically at-risk students: An open source analytics initiative. *Journal of Learning Analytics*, 1(1), 6–47.
- Jia, F., Sun, D., & Looi, C.-k. (2023). Artificial intelligence in science education (2013–2023): Research trends in ten years. *Journal of Science Education and Technology*, 1–24.
- Kallia, M. (2017). Assessment in computer science courses: A literature review. *Royal Society*, 1–60.
- Kang, J. W., Yu, Q., Golen, E., & Holden, E. P. (2018). IT curriculum: coping with technology trends & industry demands. In *Proceedings of the 19th annual SIG conference on information technology education* (pp. 44–49).
- Karakas, A., & Helic, D. (2023). Relation between student characteristics, git usage and success in programming courses. In *European conference on technology enhanced learning* (pp. 133–148). Springer.
- Kaur, A., & Chahal, K. K. (2023). A learning analytics dashboard for data-driven recommendations on influences of non-cognitive factors in introductory programming. *Education and Information Technologies*, 1–36.
- Khalil, M., & Ebner, M. (2016). What is learning analytics about? A survey of different methods used in 2013–2015. arXiv preprint arXiv:1606.02878.
- Khan, A., & Ghosh, S. K. (2021). Student performance analysis and prediction in classroom learning: A review of educational data mining studies. *Education and Information Technologies*, 26(1), 205–240.
- Khosravi, H., Shum, S. B., Chen, G., Conati, C., Tsai, Y.-S., Kay, J., Knight, S., Martinez-Maldonado, R., Sadiq, S., & Gašević, D. (2022). Explainable artificial intelligence in education. *Computers and Education: Artificial Intelligence*, 3, Article 100074.
- Kilroy, A. (2022). Guide to the highest dropout rates by major. <https://lanterncredit.com/student-loans/computer-science-dropout-rate>.
- Kirkpatrick, M. S., & Mayfield, C. (2017). Evaluating an alternative CS1 for students with prior programming experience. In *Proceedings of the 2017 ACM SIGCSE technical symposium on computer science education* (pp. 333–338).
- Kitchenham, B., Charters, S., et al. (2007). Guidelines for performing systematic literature reviews in software engineering.
- Koucham, F., El Idrissi, Y. E. B., & Ait Lahcen, A. (2022). A brief review of using LMS in computer science learning.
- Kulik, J. A., & Fletcher, J. D. (2016). Effectiveness of intelligent tutoring systems: a meta-analytic review. *Review of Educational Research*, 86(1), 42–78.
- Leitner, P., Khalil, M., & Ebner, M. (2017). Learning analytics in higher education—a literature review. *Learning Analytics: Fundaments, Applications, and Trends*, 1–23.
- Li, C., Herbert, N., Yeom, S., & Montgomery, F. (2022). Retention factors in STEM education identified using learning analytics: a systematic review. *Education Sciences*, 12(11), 781.
- Li, K. C., Wong, B. T.-M., & Chan, H. T. (2023). Predictive analytics for university student admission: a literature review. In *International conference on blended learning* (pp. 250–259). Springer.
- Li, Y., Zheng, X., Xie, F., Ye, L., Bignami, E., Tandon, Y. K., Rodríguez, M., Gu, Y., & Sun, J. (2022). Development and validation of the artificial intelligence (AI)-based diagnostic model for bronchial lumen identification. *Translational Lung Cancer Research*, 11(11), 2261.
- Lin, P.-H., & Chen, S.-Y. (2020). Design and evaluation of a deep learning recommendation based augmented reality system for teaching programming and computational thinking. *IEEE Access*, 8, 45689–45699.
- Liu, Y. (2023). Leveraging the power of ai in undergraduate computer science education: Opportunities and challenges. In *2023 IEEE frontiers in education conference* (pp. 1–5). IEEE.
- Liu, R., Zenke, C., Liu, C., Holmes, A., Thornton, P., & Malan, D. J. (2024). Teaching CS50 with AI: leveraging generative artificial intelligence in computer science education. In *Proceedings of the 55th ACM technical symposium on computer science education V. 1* (pp. 750–756).
- Lokkila, E., Christopoulos, A., & Laakso, M.-J. (2022). Automatically detecting previous programming knowledge from novice programmer code compilation history. *Informatics in Education*, 22(2), 277–294.
- López-Pernas, S., & Saqr, M. (2021). Bringing synchrony and clarity to complex multi-channel data: A learning analytics study in programming education. *IEEE Access*, 9, 166531–166541.
- López-Pernas, S., Saqr, M., Gordillo, A., & Barra, E. (2023). A learning analytics perspective on educational escape rooms. *Interactive Learning Environments*, 31(10), 6509–6525.
- Machado, M. d. C., Bravo, N. F. S., Martins, A. F., Bernardino, H. S., Barrere, E., & Souza, J. F. d. (2021). Metaheuristic-based adaptive curriculum sequencing approaches: a systematic review and mapping of the literature. *Artificial Intelligence Review*, 54(1), 711–754.
- Mai, T. T., Bezbradica, M., & Crane, M. (2022). Learning behaviours data in programming education: Community analysis and outcome prediction with cleaned data. *Future Generation Computer Systems*, 127, 42–55.
- Maier, M.-I., Czibula, G., & Onet-Marian, Z.-E. (2021). Towards using unsupervised learning for comparing traditional and synchronous online learning in assessing students' academic performance. *Mathematics*, 9(22), 2870.
- Mallik, S., & Gangopadhyay, A. (2023). Proactive and reactive engagement of artificial intelligence methods for education: a review. *Frontiers in Artificial Intelligence*, 6, Article 1151391.
- Mangaraska, K., & Giannakos, M. (2018). Learning analytics for learning design: A systematic literature review of analytics-driven design to enhance learning. *IEEE Transactions on Learning Technologies*, 12(4), 516–534.
- Maniktala, M., Cody, C., Barnes, T., & Chi, M. (2020). Avoiding help avoidance: Using interface design changes to promote unsolicited hint usage in an intelligent tutor. *International Journal of Artificial Intelligence in Education*, 30, 637–667.
- Marconi, L., Aragon, R. A. M., Zoppis, I., Manzoni, S., Mauri, G., & Epifania, F. (2020). Explainable attentional neural recommendations for personalized social learning. In *International conference of the Italian association for artificial intelligence* (pp. 67–79). Springer.
- Marconi, L., & Bandini, S. (2021). Explainable attentional neural recommendations for personalized social learning. In M. Baldoni, & S. Bandini (Eds.), *Lecture notes in computer science: vol. 12414, AIxIA 2020 - advances in artificial intelligence* (pp. 66–80). Springer, Cham.
- Margolis, J., Goode, J., & Chapman, G. (2015). An equity lens for scaling: A critical juncture for exploring computer science. *ACM Inroads*, 6(3), 58–66.
- Martin, T. (2019). Review of student soft skills development using the 5ws/h approach resulting in a realistic, experiential, applied, active learning and teaching pedagogical classroom. *Journal of Behavioral and Applied Management*, 19(1), 41–57.
- McConvey, K., Guha, S., & Kuzminykh, A. (2023). A human-centered review of algorithms in decision-making in higher education. In *Proceedings of the 2023 CHI conference on human factors in computing systems* (pp. 1–15).
- McGill, M., Snow, E., Vaval, L., DeLyser, L. A., Wortel-London, S., & Thompson, A. (2022). Practitioner perspectives on covid-19's impact on computer science education among high schools serving students from lower and higher income families. *ACM Transactions on Computing Education*, 23(1), 1–31.
- Memarian, B., & Doleck, T. (2023). Fairness, accountability, transparency, and ethics (FATE) in artificial intelligence (AI) and higher education: A systematic review. *Computers and Education: Artificial Intelligence*, 5, Article 100152.
- Messaoudi, N., Moukhliiss, G., Naciri, J. K., & Bensassi, B. (2022). Machine learning algorithms for quantifying the role of prerequisites in university success. *International Journal of Modern Education and Computer Science*, 14(6), 1.
- Mihaescu, M. C., & Popescu, P. S. (2021). Review on publicly available datasets for educational data mining. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 11(3), Article e1403.
- Moraes, L. O., & Pedreira, C. E. (2021). Clustering introductory computer science exercises using topic modeling methods. *IEEE Transactions on Learning Technologies*, 14(1), 42–54.
- Morrison, B. B., Decker, A., & Margulieux, L. E. (2016). Learning loops: A replication study illuminates impact of HS courses. In *Proceedings of the 2016 ACM conference on international computing education research* (pp. 221–230).
- Morshed Fahid, F., Tian, X., Emerson, A., B. Wiggins, J., Bounajim, D., Smith, A., Wiebe, E., Mott, B., Elizabeth Boyer, K., & Lester, J. (2021). Progression trajectory-based student modeling for novice block-based programming. In *Proceedings of the 29th ACM conference on user modeling, adaptation and personalization* (pp. 189–200).

- Mouza, C., Mead, H., Alkhateeb, B., & Pollock, L. (2022). A virtual professional development program for computer science education during COVID-19. *TechTrends*, 66(3), 436–449.
- Munir, H., Vogel, B., & Jacobsson, A. (2022). Artificial intelligence and machine learning approaches in digital education: A systematic revision. *Information*, 13(4), 203.
- Murtaza, M., Ahmed, Y., Shamsi, J. A., Sherwani, F., & Usman, M. (2022). AI-based personalized e-learning systems: Issues, challenges, and solutions. *IEEE Access*, 10, 81323–81342.
- Naseem, M., Chaudhary, K., & Sharma, B. (2022). Predicting freshmen attrition in computing science using data mining. *Education and Information Technologies*, 27(7), 9587–9617.
- Nesbit, J. C., Adesope, O. O., Liu, Q., & Ma, W. (2014). How effective are intelligent tutoring systems in computer science education? In *2014 IEEE 14th international conference on advanced learning technologies* (pp. 99–103). IEEE.
- Ng, D. T. K., Lee, M., Tan, R. J. Y., Hu, X., Downie, J. S., & Chu, S. K. W. (2023). A review of AI teaching and learning from 2000 to 2020. *Education and Information Technologies*, 28(7), 8445–8501.
- Obaido, G., Agbo, F. J., Alvarado, C., & Oyelere, S. S. (2023). Analysis of attrition studies within the computer sciences. *Ieee Access*, 11, 53736–53748.
- O'Brien, L., Kanij, T., & Grundy, J. (2025). Assessing gender bias in the software used in computer science and software engineering education. *Journal of Systems and Software*, 219, Article 112225.
- O'Donoghue, K. (2023). Learning analytics within higher education: Autonomy, beneficence and non-maleficence. *Journal of Academic Ethics*, 21(1), 125–137.
- Oleson, A., Xie, B., Salac, J., Everson, J., Kivuva, F. M., & Ko, A. J. (2022). A decade of demographics in computing education research: A critical review of trends in collection, reporting, and use. In *Proceedings of the 2022 ACM conference on international computing education research-volume 1* (pp. 323–343).
- Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamsseer, L., Tetzlaff, J. M., Akl, E. A., Brennan, S. E., et al. (2021). The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ*, 372, n71.
- Paiva, J. C., Leal, J. P., & Figueira, Á. (2022). Automated assessment in computer science education: A state-of-the-art review. *ACM Transactions on Computing Education (TOCE)*, 22(3), 1–40.
- Pantelimon, F., Bologa, R., Toma, A., & Posedaru, B. (2021). The evolution of AI-driven educational systems during the COVID-19 pandemic. *Sustainability*, 13(23), 13501.
- Parker, M. C., Guzdial, M., & Engleman, S. (2016). Replication, validation, and use of a language independent CS1 knowledge assessment. In *Proceedings of the 2016 ACM conference on international computing education research* (pp. 93–101).
- Parsons, J., & Taylor, L. (2011). Improving student engagement. *Current Issues in Education*, 14(1).
- Pereira, F. D., Fonseca, S. C., Wiktor, S., Oliveira, D. B., Cristea, A. I., Benedict, A., Fallahian, M., Dorodchi, M., Carvalho, L. S., Mello, R. F., et al. (2023). Toward supporting CS1 instructors and learners with fine-grained topic detection in online judges. *IEEE Access*, 11, 22513–22525.
- Pereira, F. D., Oliveira, E., Rodrigues, L., Cabral, L., Oliveira, D., Carvalho, L., Gasevic, D., Cristea, A., Dermeval, D., & Mello, R. F. (2023). Evaluation of a hybrid AI-human recommender for CS1 instructors in a real educational scenario. In *European conference on technology enhanced learning* (pp. 308–323). Cham: Springer Nature Switzerland.
- Pereira, F. D., Rodrigues, L., Henklain, M., Freitas, H., Oliveira, D. F., Cristea, A. I., Carvalho, L., Isotani, S., Benedict, A., Dorodchi, M., et al. (2022). Towards human-AI collaboration: a recommender system to support CS1 instructors to select problems for assignments and exams. *IEEE Transactions on Learning Technologies*.
- Pirker, J., Dengel, A., Holly, M., & Safikhani, S. (2020). Virtual reality in computer science education: A systematic review. In *Proceedings of the 26th ACM symposium on virtual reality software and technology* (pp. 1–8).
- Plate, D., & Hutson, J. (2025). *Beyond code: Redefining programming education beyond STEM*. CRC Press.
- Prekaj, B., Velardi, P., Stilo, G., Distant, D., & Faralli, S. (2020). A survey of machine learning approaches for student dropout prediction in online courses. *ACM Computing Surveys*, 53(3), 1–34.
- Qu, Y., Cai, R., & Haj-Hussein, M. (2019). Research and practice of applying adaptive learning in computer science and IT degree programs. In *2019 IEEE frontiers in education conference* (pp. 1–8). IEEE.
- Qusheh, U. B., Oyelere, S. S., Akcapinar, G., Kaliisa, R., & Laakso, M.-J. (2024). Unleashing the power of predictive analytics to identify at-risk students in computer science. *Technology, Knowledge and Learning*, 29(3), 1385–1400.
- Rachha, A., & Seyam, M. (2023). Explainable AI in education: Current trends, challenges, and opportunities. In *SoutheastCon 2023*, (pp. 232–239). IEEE.
- Raees, M., Meijerink, I., Lykourantzou, I., Khan, V.-J., & Papangelis, K. (2024). From explainable to interactive AI: A literature review on current trends in human-AI interaction. *International Journal of Human-Computer Studies*, 189, Article 103301.
- Randolph, J. J., Julnes, G., Sutinen, E., & Lehman, S. (2008). A methodological review of computer science education research. *Journal of Information Technology Education: Research*, 7(1), 135–162.
- Real, E. M., Pimentel, E. P., de Oliveira, L. V., Braga, J. C., & Stiubiener, I. (2020). Educational process mining for verifying student learning paths in an introductory programming course. In *2020 IEEE frontiers in education conference* (pp. 1–9). IEEE.
- Repenning, A., Basawapatna, A., Assaf, D., Maiello, C., & Escherle, N. (2016). Retention of flow: Evaluating a computer science education week activity. In *Proceedings of the 47th ACM technical symposium on computing science education* (pp. 633–638).
- Rico-Juan, J. R., Sánchez-Cartagena, V. M., Valero-Mas, J. J., & Gallego, A. J. (2023). Identifying student profiles within online judge systems using explainable artificial intelligence. *IEEE Transactions on Learning Technologies*.
- Rocha, H. J. B., Tedesco, P. C. D. A. R., & Costa, E. D. B. (2023). On the use of feedback in learning computer programming by novices: a systematic literature mapping. *Informatics in Education*, 22(2), 209.
- Ross, A. K. (2025). A holistic approach to building resilient computer science pathways: Exploring gender, engagement, and attrition in computer science from academia to industry.
- Rowe, E. A., & Prior, N. (2022). Procuring algorithmic transparency. *Alabama Law Review*, 74, 303.
- Salau, L., Hamada, M., Prasad, R., Hassan, M., Mahendran, A., & Watanobe, Y. (2022). State-of-the-art survey on deep learning-based recommender systems for e-learning. *Applied Sciences*, 12(23), 11996.
- Sanusi, I. T., Oyelere, S. S., Vartiainen, H., Suhonen, J., & Tukiainen, M. (2023). A systematic review of teaching and learning machine learning in K-12 education. *Education and Information Technologies*, 28(5), 5967–5997.
- Scaradozzi, D., Screpanti, L., & Cesaretti, L. (2021). Machine learning for modelling and identification of educational robotics activities. In *2021 29th mediterranean conference on control and automation* (pp. 753–758). IEEE.
- Schmude, T., Koesten, L., Möller, T., & Tschitschek, S. (2024). Better together? The role of explanations in supporting novices in individual and collective deliberations about AI. *arXiv preprint arXiv:2411.11449*.
- Senanayake, S., Karunanayaka, K., & Ekanayake, K. (2024). Review on AI assistant systems for programming language learning in learning environments. In *2024 8th SLAAI international conference on artificial intelligence* (pp. 1–6). IEEE.
- Shaik, T., Tao, X., Dann, C., Xie, H., Li, Y., & Galligan, L. (2023). Sentiment analysis and opinion mining on educational data: A survey. *Natural Language Processing Journal*, 2, Article 100003.
- Sharma, K., & Giannakos, M. (2020). Multimodal data capabilities for learning: What can multimodal data tell us about learning? *British Journal of Educational Technology*, 51(5), 1450–1484.
- Sharma, K., Nguyen, A., & Hong, Y. (2024). Self-regulation and shared regulation in collaborative learning in adaptive digital learning environments: A systematic review of empirical studies. *British Journal of Educational Technology*.
- Siegel, A. A., Zarb, M., Anderson, E., Crane, B., Gao, A., Latulipe, C., Lovellette, E., McNeill, F., & Meharg, D. (2022). The impact of COVID-19 on the CS student learning experience: How the pandemic has shaped the educational landscape. In *Proceedings of the 2022 working group reports on innovation and technology in computer science education* (pp. 165–190).
- da Silva, L. M., Dias, L. P. S., Rigo, S., Barbosa, J. L. V., Leithardt, D. R., & Leithardt, V. R. Q. (2021). A literature review on intelligent services applied to distance learning. *Education Sciences*, 11(11), 666.
- Silva, L., Mendes, A. J., & Gomes, A. (2020). Computer-supported collaborative learning in programming education: A systematic literature review. In *2020 IEEE global engineering education conference* (pp. 1086–1095). IEEE.
- Silveira, R. F., Holanda, M., Ramos, G. N., Victorino, M., & Da Silva, D. (2022). Analysis of student performance and social-economic data in introductory computer science courses at the University of Brasília. In *2022 IEEE frontiers in education conference* (pp. 1–8). IEEE.
- Sim, T. Y., & Lau, S. L. (2022). Review on challenges and solutions in novice programming education. In *2022 IEEE international conference on computing* (pp. 55–61). IEEE.
- Simbeck, K. (2024). They shall be fair, transparent, and robust: auditing learning analytics systems. *AI and Ethics*, 4(2), 555–571.
- Sinclair, J., Butler, M., Morgan, M., & Kalvala, S. (2015). Measures of student engagement in computer science. In *Proceedings of the 2015 ACM conference on innovation and technology in computer science education* (pp. 242–247).
- Smirani, L. K., Yamani, H. A., Menzli, L. J., & Boulahia, J. A. (2022a). Using ensemble learning algorithms to predict student failure and enabling customized educational paths. *Scientific Programming*, 2022(1), Article 3805235.
- Smirani, L. K., Yamani, H. A., Menzli, L. J., & Boulahia, J. A. (2022b). Using ensemble learning algorithms to predict student failure and enabling customized educational paths. *Scientific Programming*, 2022, 1–15.
- Starks, F. D., Reeves, S. M., Rickert, J., Li, K., Couch, B., & Millunchick, J. (2024). Rethinking undergraduate computer science education: Using the 4Es heuristic to center students in an introductory computer science course. *Education Sciences*, 14(5).
- Sunil Prakash, L., Kutti, N. S., & Sajeev, A. (2010). Review of challenges in content extraction in web based personalized learning content management systems. In *Proceedings of the 12th international conference on information integration and web-based applications & services* (pp. 829–832).
- Švábenský, V., Verger, M., Rodrigo, M. M. T., Monterozo, C. J. G., Baker, R. S., Saavedra, M. Z. N. L., Lallé, S., & Shimada, A. (2024). Evaluating algorithmic bias in models for predicting academic performance of filipino students. *arXiv preprint arXiv:2405.09821*.

- Sychev, O., Penskoy, N., Anikin, A., Denisov, M., & Prokudin, A. (2021). Improving comprehension: Intelligent tutoring system explaining the domain rules when students break them. *Education Sciences*, 11(11), 719.
- Sypas, A., & Kalles, D. (2022). Reviewing process mining applications and techniques in education. *Journal of Artificial Intelligence Application*, 13, 83–102.
- Takács, R., Kárász, J. T., Takács, S., Horváth, Z., & Oláh, A. (2022). Successful steps in higher education to stop computer science students from attrition. *Interchange*, 53(3), 637–652.
- Tan, F. Z., Lim, J. Y., Chan, W. H., & Idris, M. I. T. (2024). Computational intelligence in learning analytics: A mini review. *ASEAN Engineering Journal*, 14(4), 135–151.
- The Joint Task Force on Computing Curricula (2023). Computer science curricula 2023: Version Gamma. <https://www.acm.org/education/curricula-recommendations>, Association for Computing Machinery (ACM), IEEE Computer Society (IEEE-CS), and Association for the Advancement of Artificial Intelligence (AAAI).
- Tian, X., Risha, Z., Ahmed, I., Lekshmi Narayanan, A. B., & Biehl, J. (2021). Let's talk it out: A chatbot for effective study habit behavioral change. *Proceedings of the ACM on Human-Computer Interaction*, 5(CSCW1), 1–32.
- Toll, D., & Wingkvist, A. (2017). How tool support and peer scoring improved our students' attitudes toward peer reviews. In *Proceedings of the 2017 ACM conference on innovation and technology in computer science education* (pp. 311–316).
- Toti, G., Si, L., Daniels, D., Amoozadeh, M., Alipour, M. A., & Chen, G. (2025). Students and instructors reflections on the impact of COVID-19 on computer science education after 1 year of remote teaching. *Discover Education*, 4(1), 47.
- Vakil, S. (2018). Ethics, identity, and political vision: Toward a justice-centered approach to equity in computer science education. *Harvard Educational Review*, 88(1), 26–52.
- Valiant, A. (2020). The capabilities of students in poverty: Student outcomes from a united states department of education TRIO program.
- Varghese, V. V., & Renumol, V. (2023). Video games for assessing computational thinking: a systematic literature review. *Journal of Computers in Education*, 1–46.
- Veerasamy, A. K., Laakso, M.-J., & D'Souza, D. (2022). Formative assessment tasks as indicators of student engagement for predicting at-risk students in programming courses. *Informatics in Education*, 21(2), 375–393.
- Venkatesan, R. G., Karmegam, D., & Mappillairaju, B. (2024). Exploring statistical approaches for predicting student dropout in education: a systematic review and meta-analysis. *Journal of Computational Social Science*, 7(1), 171–196.
- Verdú, E., Regueras, L. M., Gal, E., de Castro, J. P., Verdú, M. J., & Kohen-Vacs, D. (2017). Integration of an intelligent tutoring system in a course of computer network design. *Educational Technology Research and Development*, 65, 653–677.
- Vieriu, A. M., & Petrea, G. (2025). The impact of artificial intelligence (AI) on students' academic development. *Education Sciences*, 15(3), 343.
- Wang, F. H. (2024). On the effect of explainable AI on programming learning: A case study of using gradient integration technology. In *CS & IT conference proceedings: vol. 14, no. 12, CS & IT Conference Proceedings*.
- Wang, J., & Hejazi Moghadam, S. (2017). Diversity barriers in K-12 computer science education: Structural and social. In *Proceedings of the 2017 ACM SIGCSE technical symposium on computer science education* (pp. 615–620).
- Wang, Q., & Mousavi, A. (2023). Which log variables significantly predict academic achievement? A systematic review and meta-analysis. *British Journal of Educational Technology*, 54(1), 142–191.
- Wiredu, J. K., Abuba, N. S., & Acheampong, R. W. (2024). Enhancing accessibility and engagement in computer science education for diverse learners. *Asian Journal of Research in Computer Science*, 17(10), 45–61.
- Wong, B. T.-M., Li, K. C., & Choi, S. P.-M. (2018). Trends in learning analytics practices: a review of higher education institutions. *Interactive Technology and Smart Education*, 15(2), 132–154.
- Wu, F., Dang, Y., & Li, M. (2025). A systematic review of responses, attitudes, and utilization behaviors on generative AI for teaching and learning in higher education. *Behavioral Sciences*, 15(4), 467.
- Xu, J., Wei, T., & Lv, P. (2022). SQL-DP: A novel difficulty prediction framework for SQL programming problems. *International Educational Data Mining Society*.
- Yee-King, M., McCallum, L., Llano, M. T., Ruzicka, V., d'Inverno, M., & Grierson, M. (2020). Examining student coding behaviours in creative computing lessons using abstract syntax trees and vocabulary analysis. In *Proceedings of the 2020 ACM conference on innovation and technology in computer science education* (pp. 273–279).
- Yuan, H., & Cao, P. (2019). Collaborative assessments in computer science education: A survey. *Tsinghua Science and Technology*, 24(4), 435–445.
- Yusuf, A., Noor, N. M., & Bello, S. (2023). Using multimodal learning analytics to model students' learning behavior in animated programming classroom. *Education and Information Technologies*, 1–44.
- Zhong, L. (2015). A systematic overview of learning analytics in higher education. *Journal of Educational Technology Development and Exchange (JETDE)*, 8(2), 3.
- Zhu, M., & Zhang, K. (2025). Artificial intelligence for computer science education in higher education: A systematic review of empirical research published in 2003–2023: M. Zhu, K. Zhang. *Technology, Knowledge and Learning*, 1–25.
- Zoppis, I., Manzoni, S., Mauri, G., Aragon, R. A. M., Marconi, L., & Epifania, F. (2020). Attentional neural mechanisms for social recommendations in educational platforms.. In *CSEDU (1)* (pp. 111–117).